Mr. Rohit Pratap Singh – Notes + Practical (practicals are divided into sub-practicals (expandable format)

1.1 Introduction to Web Technology

1.2 Web Designing & Development

1.3 Client-Server Architecture

1.4 Introduction to Text Editors

1.5 Installation & Setup of VS Code Editor

Unit 1

1.1 Introduction to Web Technology

1.1.1 What is the web?

The web, also known as the World Wide Web (WWW).
The Web (World Wide Web) is a system of websites and webpages that can be accessed through the Internet using browsers.

1.1.2 Evolution of Web

Web 1.0: Static, read-only websites.
Web 2.0: Interactive, user-driven content (social media, blogs).
Web 3.0: AI-driven, decentralized, blockchain-based, semantic web.

1.1.3 Some web components

Web – Page

A document which can be displayed in a web browser such as Firefox, Google Chrome, Opera, Microsoft Internet Explorer or Edge, or Apple's Safari.

Web-Site

Collection of web pages which are grouped together and usually connected together in various ways.

Web Server

A web server is a computer that runs websites.

A web server is a software that Serves web content through the HTTP protocol

1.1.4 Definition of Web Technology

"Web Technology is the technology used to make and run websites on the Internet."

"Web Technology refers to the collection of tools, languages, and protocols that allow computers and devices to communicate through the World Wide Web (WWW)."

These technologies allow users to browse websites, interact with online content, and share information over the internet.

1.1.5 Components of Web Technology

1. HTML (Hypertext Markup Language)

- HTML is the foundational markup language of the web.
- It defines the structure of web pages by specifying elements such as headings, paragraphs, lists, images, and links.
- HTML acts as the starting point for any website or web application.

2. CSS (Cascading Style Sheets):

- While HTML defines the structure, CSS is responsible for the appearance.
- It controls the design, including colors, fonts, layout, and responsiveness, ensuring that websites look good across various devices and screen sizes.

3. JavaScript:

- JavaScript is a programming language that adds interactivity to websites.
- It enables dynamic content, such as clickable buttons, animations, and real-time form validation.
- JavaScript is often used alongside HTML and CSS to create seamless user experiences.

4. Web Browsers:

- Browsers like Google Chrome, Mozilla Firefox, and Safari interpret HTML, CSS, and JavaScript and render websites on users' devices.
- They play a crucial role in ensuring a smooth user experience.

5. Web Servers:

- Web servers host websites and serve content over the internet.
- They store files and data that make up a website and deliver them to users when requested.
- Web servers use protocols like **HTTP** (Hypertext Transfer Protocol) and **HTTPS** (HTTP Secure) to communicate with browsers.

6. Web Protocols:

- Protocols are the rules that govern communication between web servers and browsers.
- **HTTP** is the most common protocol, but **HTTPS,** which encrypts data for security, is becoming more prevalent.

- Newer protocols like **HTTP/2** and **HTTP/3** offer faster and more efficient communication.

7. APIs (Application Programming Interfaces):

- APIs are essential for modern web applications.
- They allow different software systems to communicate with each other.
- **Example**, a weather website may use an API to fetch real-time data from a third-party weather service.

8. Databases:

- Databases store and manage data for websites and applications.
- They allow for efficient storage, retrieval, and updating of data.
- Examples of databases used in web technology include **MySQL, MongoDB**, and **PostgreSQL**.

1.1.6 Types of Web Technologies

1. Frontend Technologies

Frontend technologies are used to build the user-facing part of a website or web application. They include:

- **HTML, CSS, and JavaScript:** The trio of core languages for designing and developing user interfaces.
- **Frontend Frameworks:** Tools like **React, Vue.js**, and **Angular** help developers quickly build complex, interactive user interfaces and manage state.

2. Backend Technologies

Backend technologies are used to build and manage the server-side of web applications. These include:

- **Server-side Languages:** Python, Ruby, PHP, and **Node.js** are popular backend programming languages used to handle business logic, manage databases, and process user requests.
- **Database Management Systems: MySQL, MongoDB, and PostgreSQL** are commonly used to store and retrieve data on the server-side.
- **Web Servers: Apache, Nginx,** and cloud services like **AWS** and **Google Cloud** host and manage websites.

3. Full-Stack Technologies

Full-stack development involves both frontend and backend development. Frameworks like **Django** (Python), **Ruby on Rails** (Ruby), and **Node.js** (JavaScript) allow developers to handle both client-side and server-side development.

4. Web APIs

Web APIs enable communication between different software components. Some popular API types include:

- **RESTful APIs:** Representational State Transfer (REST) APIs are widely used for client-server communication.
- **GraphQL:** A modern alternative to REST APIs, allowing clients to request only the data they need.
- **WebSockets:** WebSockets allow for real-time, two-way communication between the client and server.

1.2 Web Designing & Development

1.2.1 Web Design

It involves the color scheme, layout, information flow, and all aspects of UI/UX (user interface and user experience).

Web design covers everything related to a website's visual aesthetics and usability

- **Adobe Creative Cloud.** Software like Photoshop and Illustrator help designers create design elements for web pages, such as graphics and logos.
- **Graphic design.** Graphic design involves crafting visuals and layouts to make the website visually appealing and enhance the overall web experience.
- **UI/UX design.** UI/UX design focuses on creating intuitive user interfaces and seamless user experiences by prioritizing ease of use and navigation.
- **Figma.** This browser-based design tool helps designers do collaborative design work and prototyping, as well as create high-fidelity mock-ups.

1.2.2 Web Development

Front-end developers

Front-end developers focus on what users interact with on the website. They use HTML, CSS, and JavaScript to bring web designs to life. Key tools and technologies include:

- CSS preprocessors like LESS or Sass for efficient style management.
- JavaScript frameworks such as AngularJS, React, Ember.js, and Vue.js have revolutionized how developers build dynamic and responsive web applications.
- Libraries like jQuery.
- Git and GitHub for version control.

They also use on-site search engine optimization (SEO) to make sure the website ranks high in search engine results.

Back-end developers

Back-end developers handle the data and server-side functions of web applications. Essential tools and skills include:

- Programming languages like PHP, Python, Java, and C#.
- Frameworks such as Ruby on Rails, Symfony, and .NET that streamline building robust server-side applications.
- Database management systems, including MySQL, MongoDB, and PostgreSQL.
- Security and authentication technologies such as OAuth and Passport for protecting data integrity and user privacy.

Front-end developer	**Back-end developer**
It is Part of website User Can see Code and Interact it.	It is brain of website that Code never visible to End User
Run on the client side browser	Run on the server side browser
HTML, CSS, JavaScript (and frameworks like Bootstrap, React).	PHP, Python, Java, Node.js and Databases like MySQL, MongoDB.
Designs the layout, look and feel of the website.	Handles the logic, database, and server operations.
Limited Security	Advanced Security

1.3 Client–Server Architecture

1.3.1 Definition & Concept

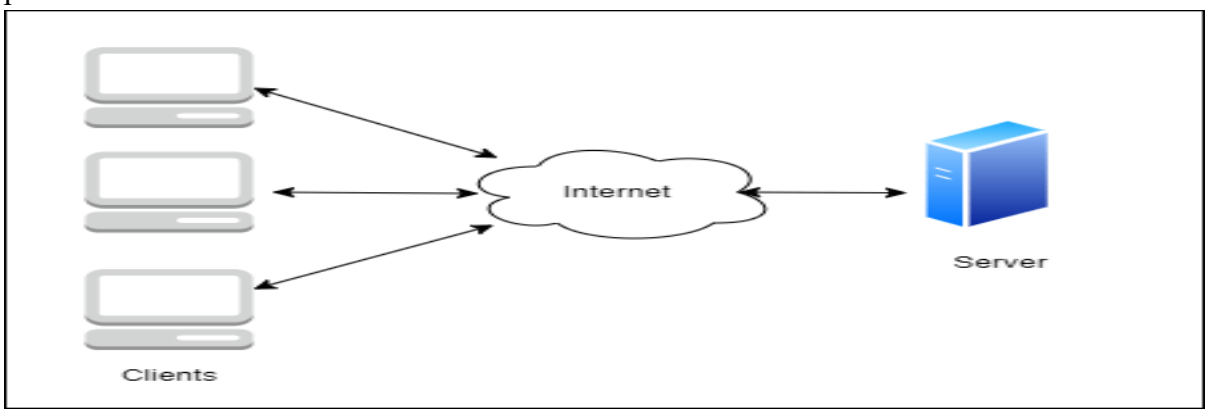
Client-Server Architecture is a model where tasks are divided between two entities:

(i)Client = Request

(ii)Server = Response

A computing model where the client requests services and the server processes and responds.

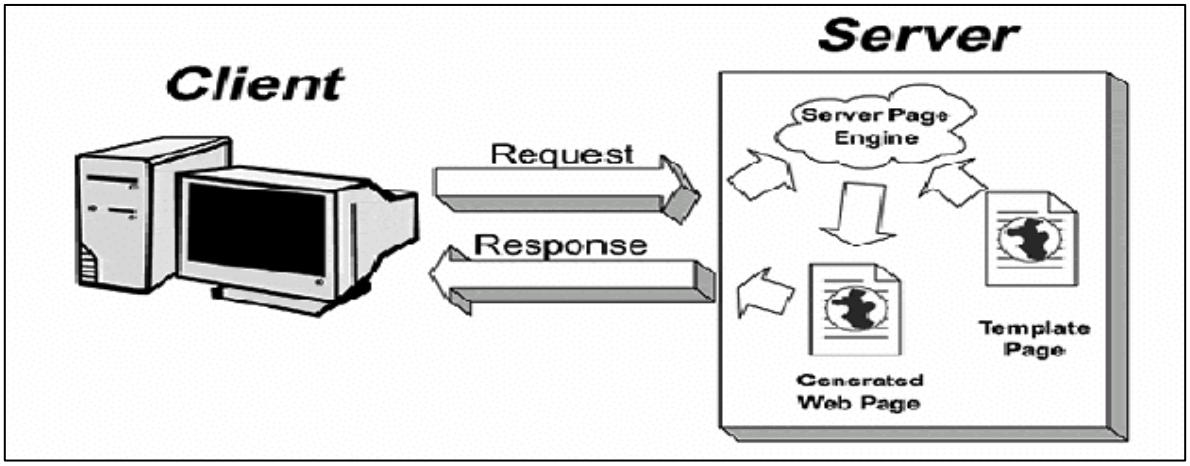
Client-Server Model is a distributed architecture where clients request services and servers provide them



a) **Client:** An application that requests services from the server, such as data retrieval, storage, calculations, or other functions.

b) **Server**: An application that processes client requests, sends responses, or performs specified actions. The server and client may reside on the same machine or different devices across the network. Effective Communication occurs via predefined protocols like HTTP, FTP, or SMTP.

1.3.2 How client requests and server responds



1.3.3 Types of Client-Server Architecture

1. 2-tier Architecture

Client directly communicates with Server.

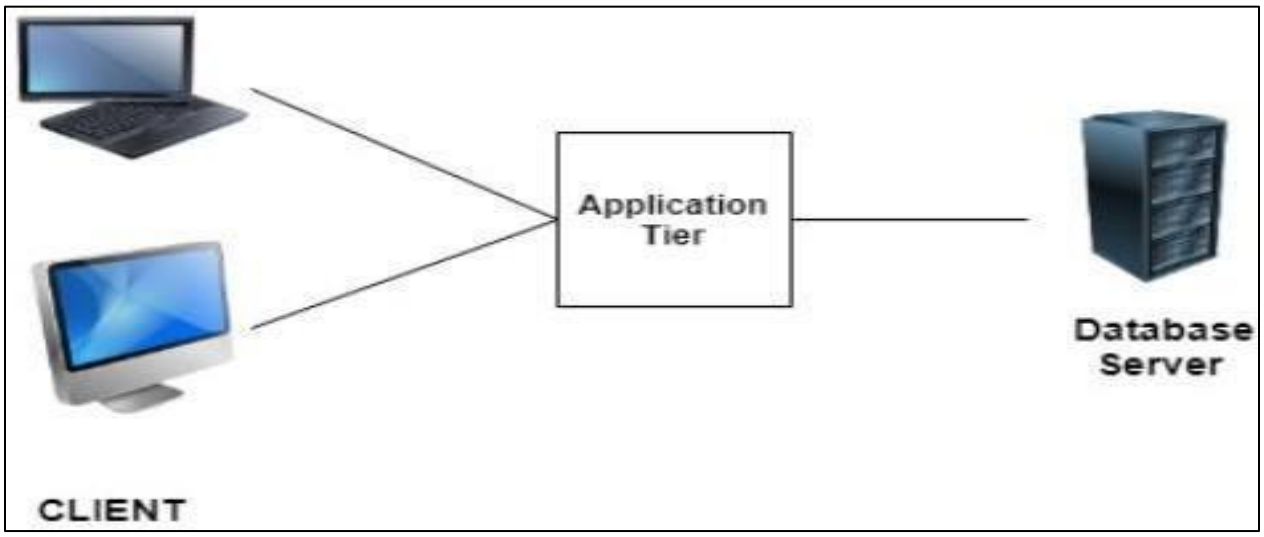
Two tier architecture primarily has two parts, a client tier and a server tier. The client tier sends a request to the server tier and the server tier responds with the desired information.

2.3-tierArchitecture

Three tier architecture has three layers namely client, application and data layer.

Client → Application Server → Database.

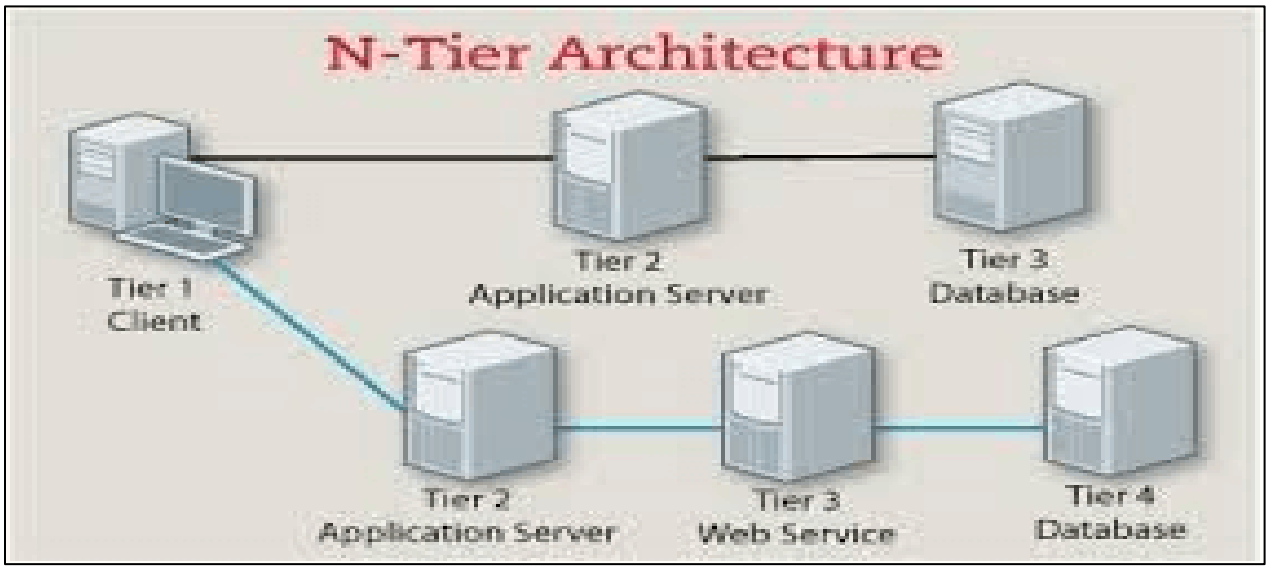
client layer is the one that requests the information. In this case it could be the GUI, web interface etc. The application layer acts as an interface between the client and data layer. It helps in communication and also provides security. The data layer is the one that actually contains the required data.



3. N-tier Architecture

N-tier architecture is also called Distributed Architecture or Multi-tier Architecture. Multiple layers of servers for scalability.

It is similar to three-tier architecture but the number of the application server is increased and represented in individual tiers



1.4. Introduction to Text Editors

A Text Editor is software used to write and edit code.

Text editor is a program that lets the user edit text. Whenever you're able to type some text in the input, you are editing the text, and that input is somehow a text editor. By this definition, we can call the HTML input element a basic text editor.

1.4.1 Basic text editors

The simplest form of text editors is the basic one. In a basic text editor, you can create and edit plain text without any styling or formatting.

- TextEdit (Mac)
- Notepad (Windows)
- Nano (Linux)
- HTML textarea (Web)

1.4.2 Integrated Development Environments

IDEs are some sort of special text editors that are designed for the programming and software development purposes. Syntax highlighting, intelligence and code completion, version controlling and debugging are the benefits of integrated development environments.

- Visual Studio
- Eclipse
- Jet Brains Products like PyCharm and WebStorm

1.4.3 Difference between Text Editor & IDE

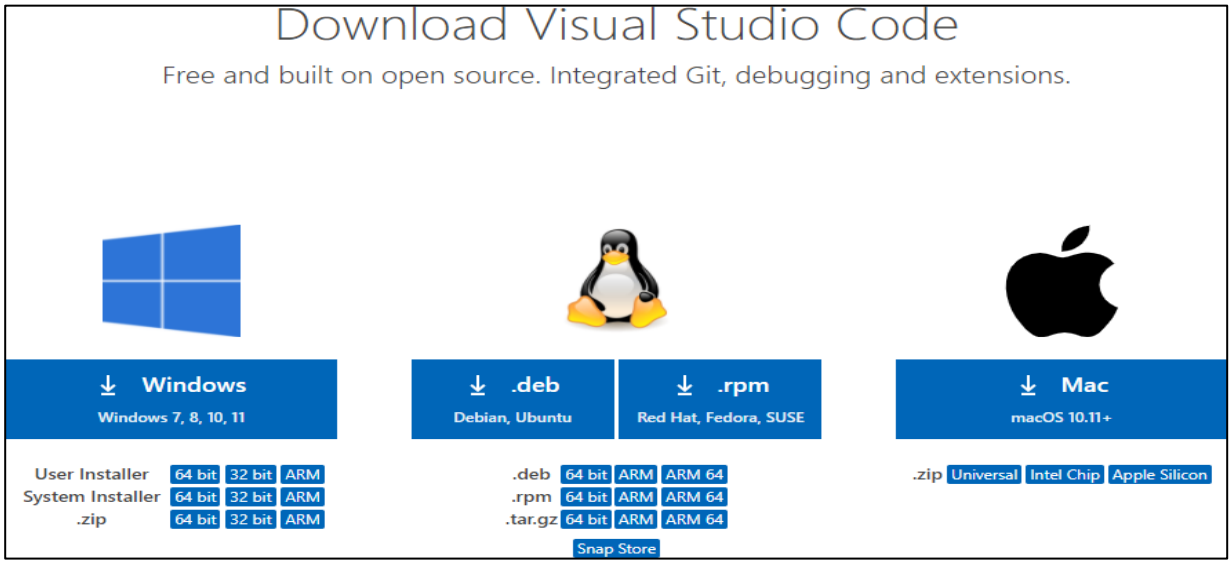
Text Editor: Lightweight, mainly for writing code (Notepad, Sublime, Atom).
- **IDE:** Advanced, includes debugging, compilation, version control (VS Code, IntelliJ).

Feature	**Text Editor**	**IDE (Integrated Development Environment)**
Definition	A simple tool to write and edit plain text or code.	A complete software suite that provides tools for writing, testing, and debugging code.
Main Purpose	Only for writing and editing text/code.	For software development (coding, compiling, debugging, project management).
Complexity	Lightweight and simple.	Heavy, feature-rich, and advanced.
Examples	Notepad, Notepad++, Sublime Text, VS Code (when used just for editing).	Eclipse, PyCharm, IntelliJ IDEA, Visual Studio, Android Studio.
Features	Syntax highlighting, basic editing.	Code editor + Debugger + Compiler/Interpreter + Build automation + Version control integration.
Usage	Best for small coding tasks or quick edits.	Best for large projects and full application development.
Resource Requirement	Low system resources (RAM, CPU).	Requires more system resources.

1.5 Installation & Setup of VS Code Editor

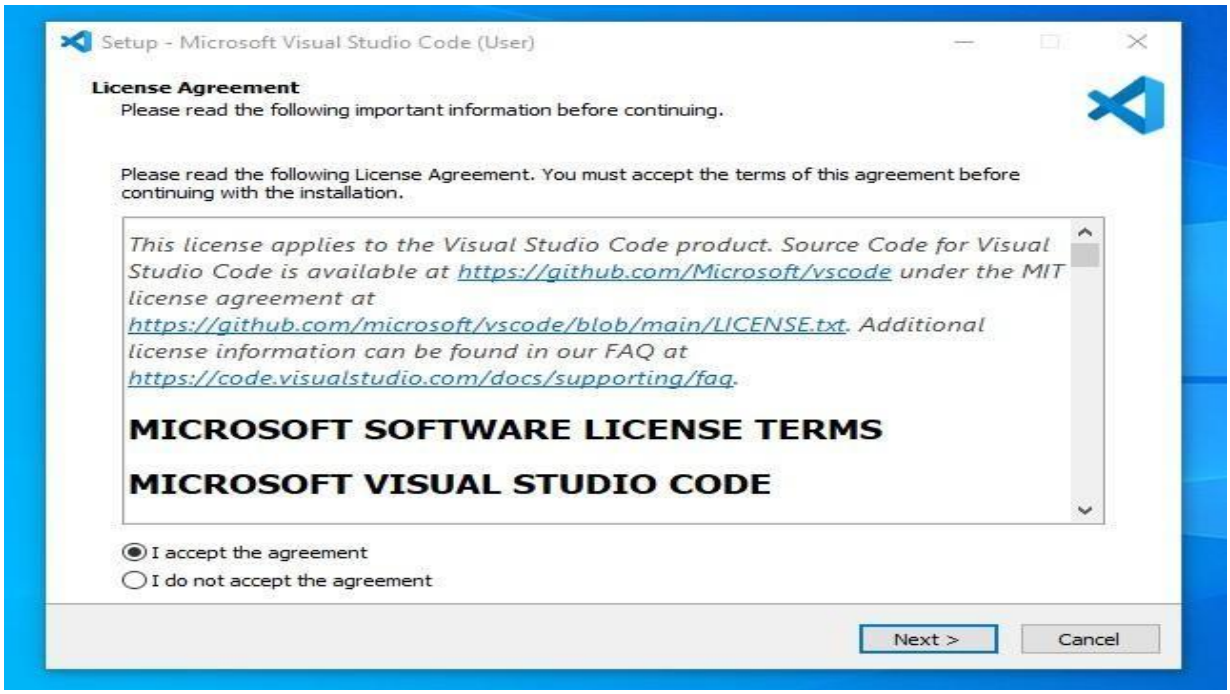
Step 1: Downloading VS Code (from official site)
Visit https://code.visualstudio.com/ and download the installer.

Press the "**Download for Windows**" button on the website to start the download of the Visual Studio Code Application.



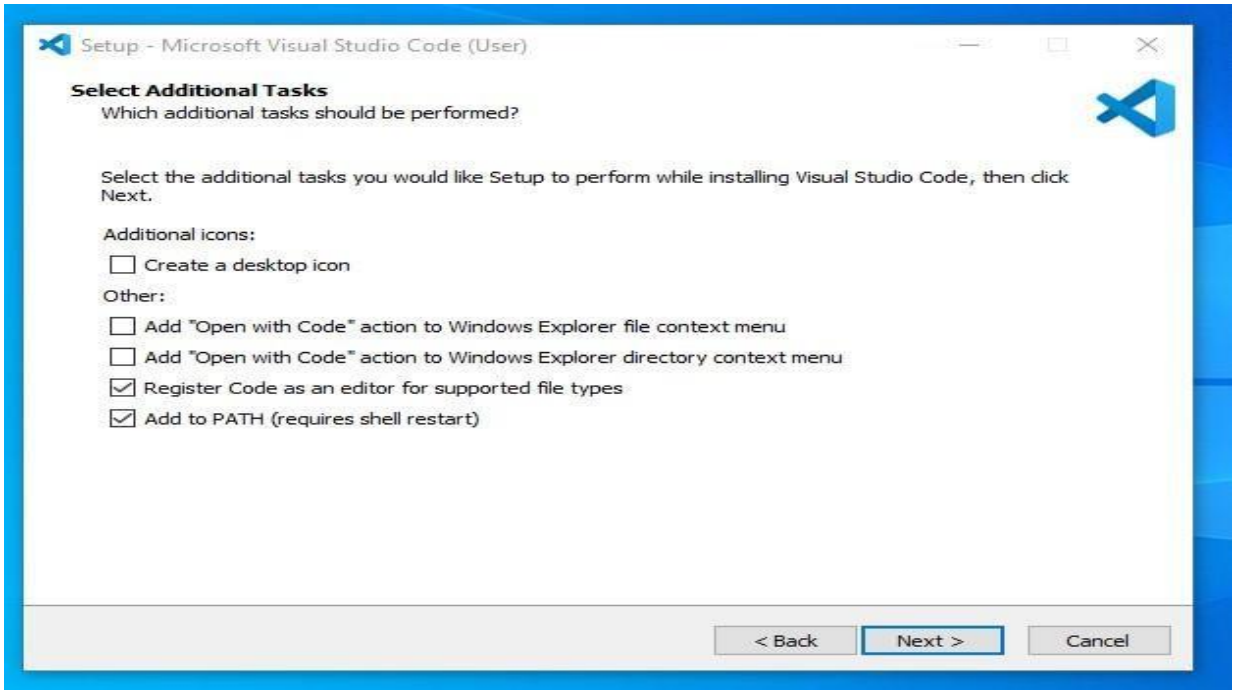
Step 2: Installation process (Windows/Linux/Mac)
Run the installer → Accept License → Choose location → Install.

After the Installer opens, it will ask you to accept the terms and conditions of the Visual Studio Code. Click on **I accept the agreement** and then click the **Next** button.



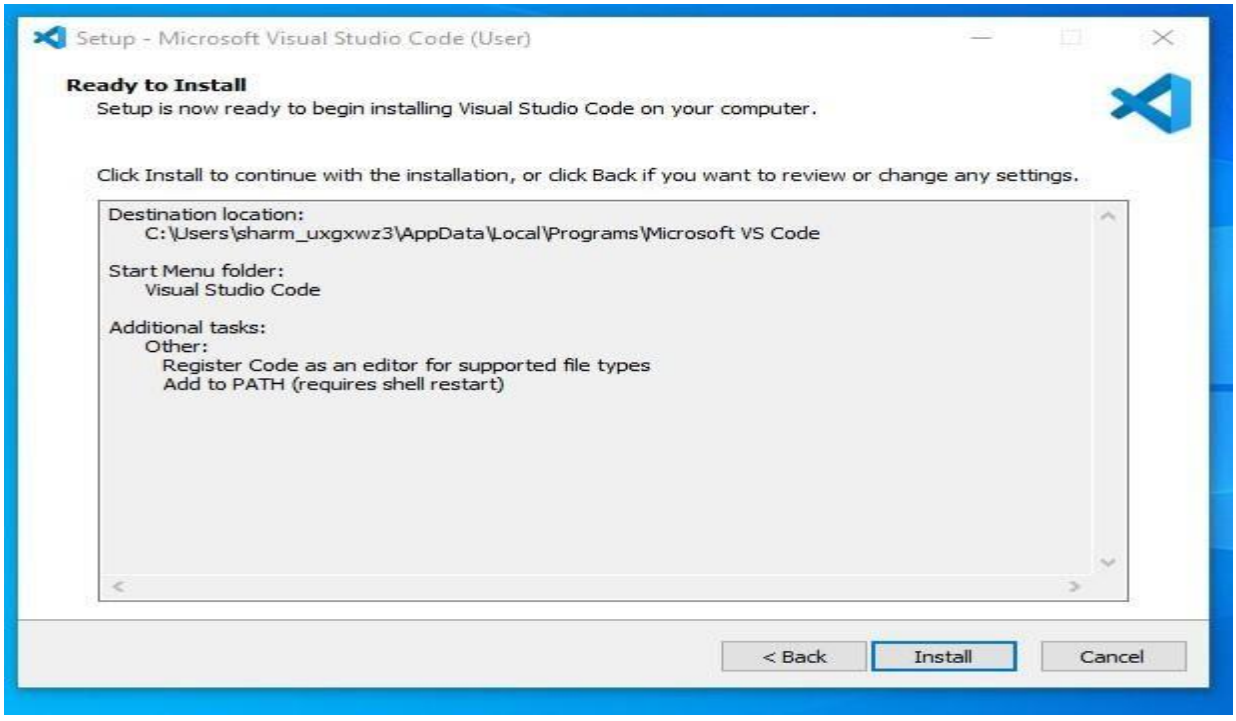
Step 3: Choose the location data for running the Visual Studio Code

Choose the location data for running the Visual Studio Code. It will then ask you to browse the location. Then click on the **Next** button.



Step 4: installation setup.

Then it will ask to begin the installation setup. Click on the **Install** button.

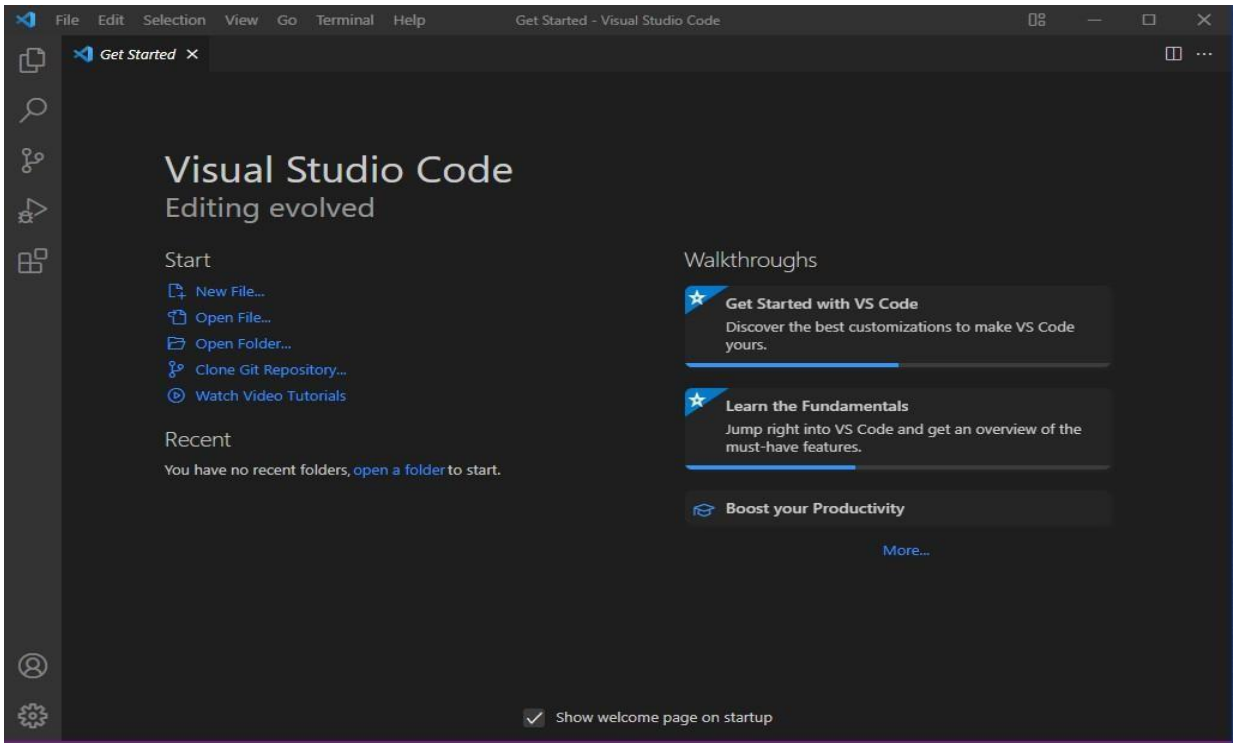


Step 5: Launch Visual Studio Code

After the Installation setup for Visual Studio Code is finished, it will show a window like this below. Tick the "**Launch Visual Studio Code**" checkbox and then click **Next**.

Step 6: Start Your Coding in Visual Studio

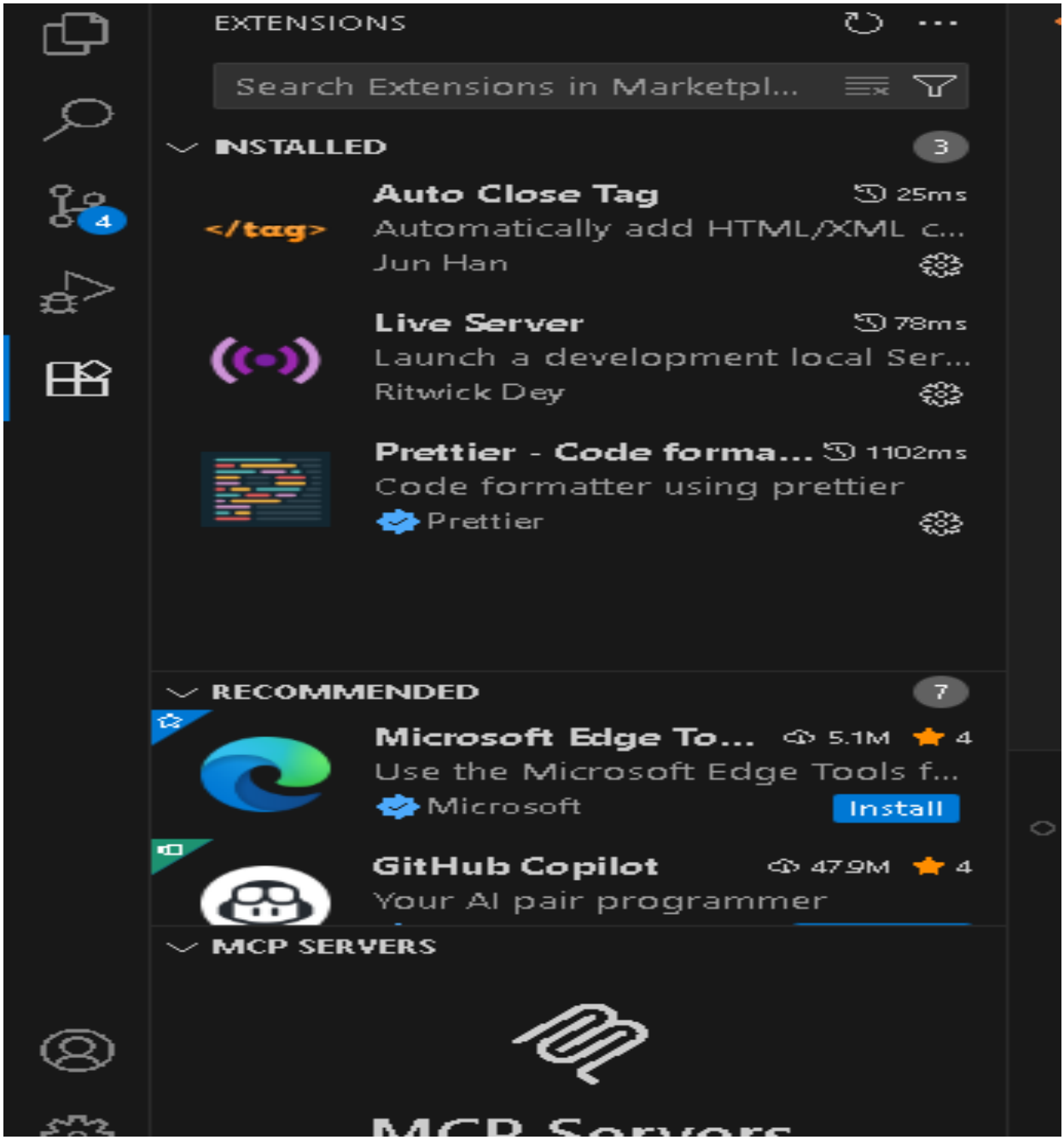
After the previous step, the Visual Studio Code window opens successfully. Now you can create a new file in the Visual Studio Code window and choose a language of yours to begin your programming journey!



1.5.1 How to Add Extensions in VS Code

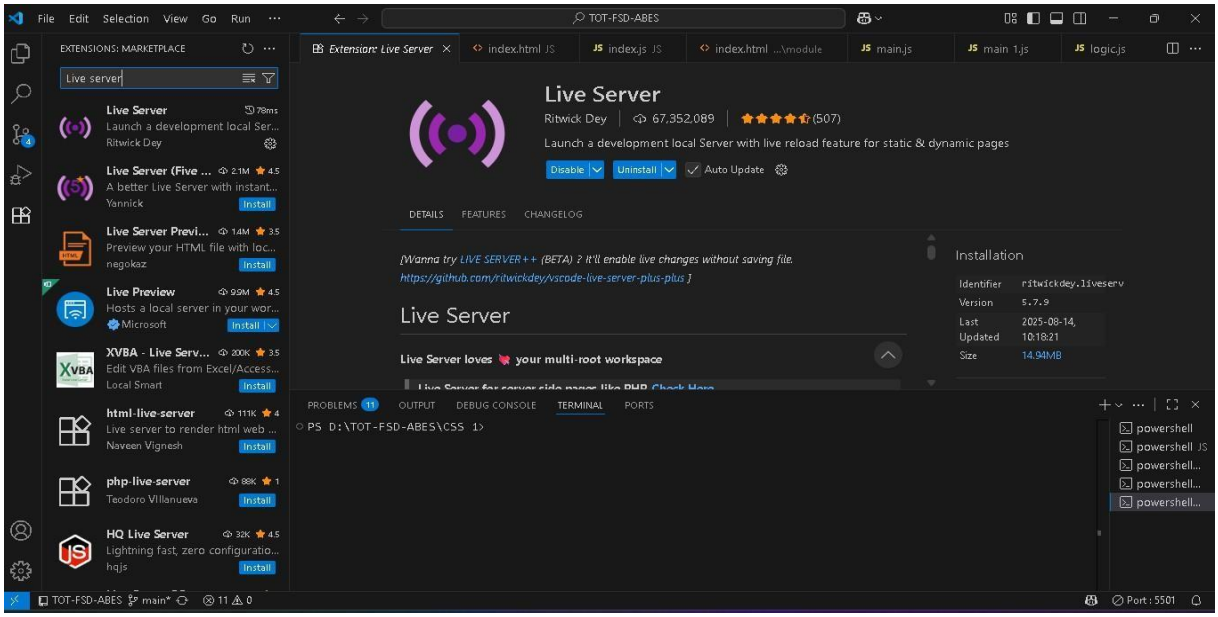
Step 1: Open Extensions Panel

- Open Visual Studio Code.
- On the left sidebar, click the Extensions icon (looks like 4 small squares).
- Shortcut: Ctrl + Shift + X



Step 2: Search for an Extension

- At the top of the Extensions panel, type the name of the extension you want.
- Example: Python, C/C++, Prettier - Code Formatter, Live Server



Step 3: Install the Extension

- From the search results, click the Install button next to the extension.
- It will download and install automatically.

Step 4: Reload if Needed

- Some extensions require a reload of VS Code.
- If prompted, click Reload.

Step 5: Manage Extensions

- Go to the Installed tab in the Extensions panel.
- From here you can:
    - Enable / Disable
    - Update
    - Uninstall

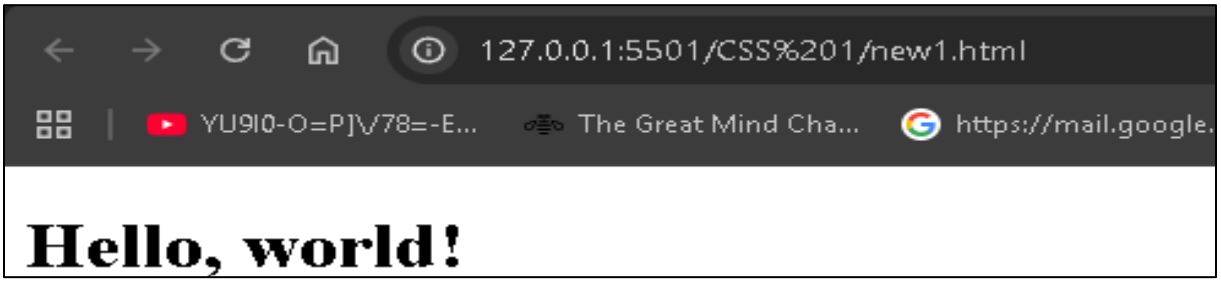
First sample HTML Program in VS Code:

- Open **Visual Studio Code.**
- Create a **new folder** (for example: MyFirstHTML).
- Inside the folder, create a new file and save it as: **index.html**

```
<!DOCTYPE html>
<html>
<head>
  <title>My First Web Page</title>
</head>
<body>
  <h1>Hello, world!</h1>
</body>
</html>
```

Output

Output: Displays 'Hello, Web Technology!' in browser.



Second HTML Program in VS Code

- Open **Visual Studio Code**.
- Create a **new folder** (for example: MySecondHTML).
- Inside the folder, create a new file and save it as:
  New.html

```
<!DOCTYPE html>
<html>
<head>
  <title>College Info</title>
</head>
<body>
  <h1>Rohit Pratap Singh</h1>
  <h2>ABES Engineering College </h2>
  <h3>CSE-AIML Department</h3>
</body>
</html>
```

Output

ASSIGNMENT -1

1. **Define Web Technologies. Explain different types of Web Technologies with suitable examples.**
2. **Difference between Front-End Developer and Back-End Developer.**
3. **Explain Client-Server Architecture with diagram**
4. **Create a simple HTML webpage in VS Code with the following details:**

   linstructions:

   1. **Create a new folder named Web-Assignment on your computer.**
   2. **Inside the folder, create a file named index.html.**
   3. **The webpage must include the following:**
      - **A title in the browser tab: *"My first HTML Page"*.**
      - **A main heading (H1) containing your Name, Father's Name, and Mother's Name.**
      - **A subheading (H2) mentioning your Branch and Section.**
      - **A paragraph (P) that contains your address in 3–4 lines.**
   4. **Save the file and run it in a web browser to check the output.**

Solution:

1.Define Web Technologies. Explain different types of Web Technologies with suitable examples.

Web Technology is the technology used to make and run websites on the Internet.

Web Technology refers to the collection of tools, languages, and protocols that allow computers and devices to communicate through the World Wide Web (WWW)."

These technologies allow users to browse websites, interact with online content, and share information over the internet.

Types of Web Technologies

Frontend Technologies

Frontend technologies are used to build the user-facing part of a website or web application. They include:

- **HTML, CSS, and JavaScript:** The trio of core languages for designing and developing user interfaces.
- **Frontend Frameworks:** Tools like **React, Vue.js**, and **Angular** help developers quickly build complex, interactive user interfaces and manage state.

Backend Technologies

Backend technologies are used to build and manage the server-side of web applications. These include:

- **Server-side Languages:** Python, Ruby, PHP, and **Node.js** are popular backend programming languages used to handle business logic, manage databases, and process user requests.
- **Database Management Systems: MySQL, MongoDB, and PostgreSQL** are commonly used to store and retrieve data on the server-side.
- **Web Servers: Apache, Nginx,** and cloud services like **AWS** and **Google Cloud** host and manage websites.

Full-Stack Technologies

Full-stack development involves both frontend and backend development. Frameworks like **Django** (Python), **Ruby on Rails** (Ruby), and **Node.js** (JavaScript) allow developers to handle both client-side and server-side development.

Web APIs

Web APIs enable communication between different software components. Some popular API types include:

- **RESTful APIs:** Representational State Transfer (REST) APIs are widely used for client-server communication.
- **GraphQL:** A modern alternative to REST APIs, allowing clients to request only the data they need.
- **WebSockets:** WebSockets allow for real-time, two-way communication between the client and server.

2.Difference between Front-End Developer and Back-End Developer.

Front-end developer	**Back-end developer**
It is Part of website User Can see Code and Interact it.	It is brain of website that Code never visible to End User
Run on the client side browser	Run on the server side browser
HTML, CSS, JavaScript (and frameworks like Bootstrap, React).	PHP, Python, Java, Node.js and Databases like MySQL, MongoDB.
Designs the layout, look and feel of the website.	Handles the logic, database, and server operations.
Limited Security	Advanced Security

3.Explain Client-Server Architecture with diagram

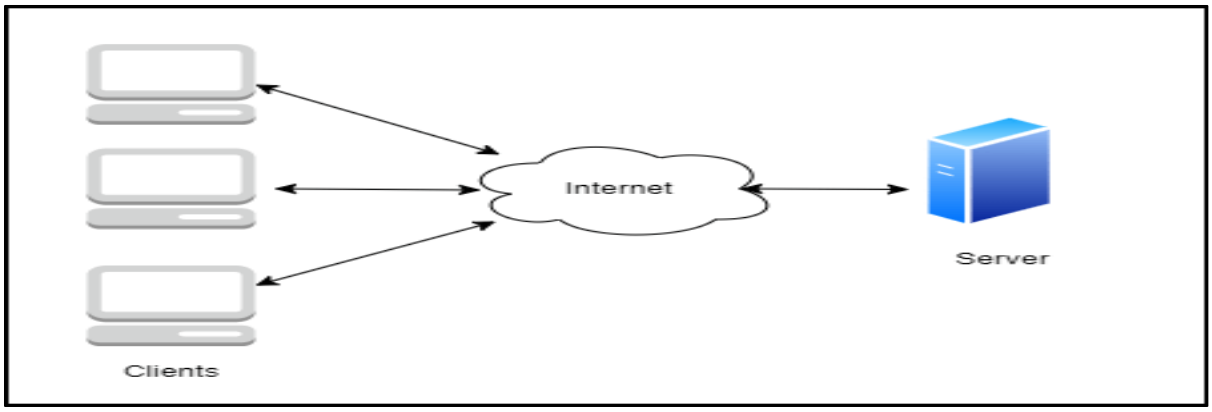
Client-Server Architecture is a model where tasks are divided between two entities:

(i)Client = Request

(ii)Server = Response

A computing model where the client requests services and the server processes and responds.

Client-Server Model is a distributed architecture where clients request services and servers provide them



Client: An application that requests services from the server, such as data retrieval, storage, calculations, or other functions.

Server: An application that processes client requests, sends responses, or performs specified actions. The server and client may reside on the same machine or different devices across the network. Effective Communication occurs via predefined protocols like HTTP, FTP, or SMTP.

4.Create a simple HTML webpage in VS Code with the following details

Instructions:

a. Create a new folder named **Web_Assignment** on your computer.

b. Inside the folder, create a file named **index.html**.

c. The webpage must include the following:

i. A **title** in the browser tab: *"My first HTML Page"*.

ii. A **main heading (H1)** containing your **Name, Father's Name, and Mother's Name**.

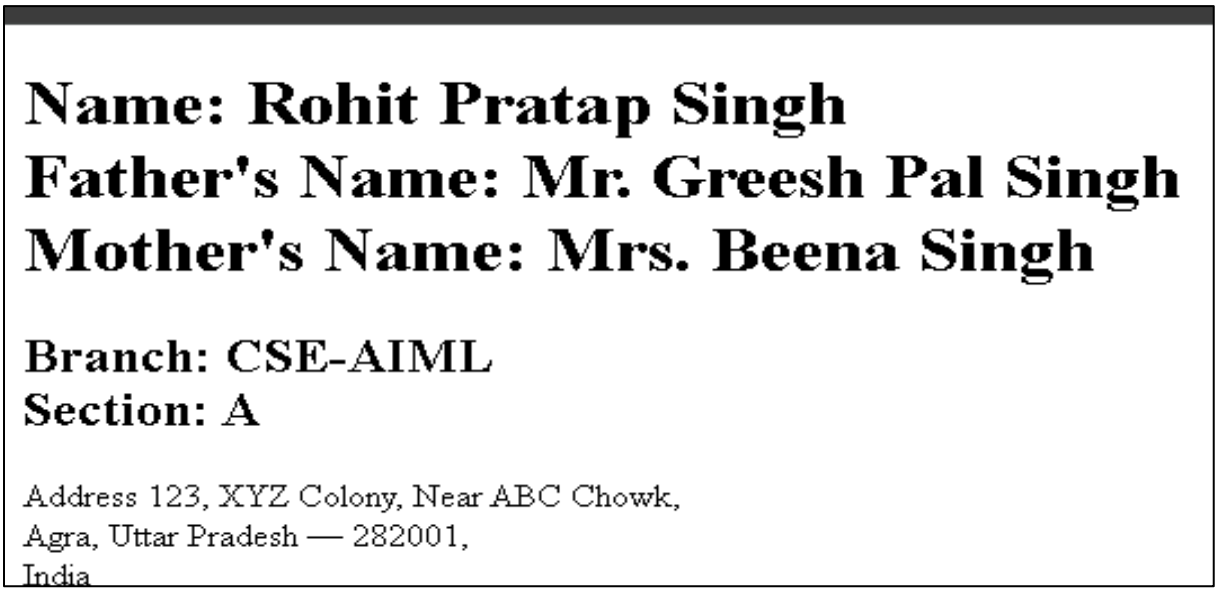
iii. A **subheading (H2)** mentioning your **Branch and Section**

iv. A **paragraph (P)** that contains your **address** in 3–4 lines.

d. Save the file and **run it in a web browser** to check the output

```
<!DOCTYPE html>
<html lang="en">
<head>
<title>My first HTML Page</title>
</head>
<h1>
Name: Rohit Pratap Singh<br />
Father's Name: Mr. Greesh Pal Singh<br />
Mother's Name: Mrs. Beena Singh
 </h1>
<h2>
Branch: CSE-AIML<br />
Section: A
</h2>
<p>
 Address 123, XYZ Colony, Near ABC Chowk,<br />
            Agra, Uttar Pradesh — 282001, <br />
India
</p>
</body>
</html>
```

Output



Ms. Pranshi Verma – Notes + Practical + Video (practicals are divided into sub-practicals (expandable format)

1.6 Introduction of Git & GitHub

1.7 Installation & Configuration of Git Account

1.6 Introduction of Git & GitHub

What is a Version Control System (VCS)?

👉 Imagine you are writing your **Project Report**.

- Day 1: You save it as report.docx.
- Day 2: You add more content and save as report-final.docx.
- Day 3: You make changes and save as report-final2.docx.

Soon, your folder looks like this:

report.docx
report-final.docx
report-final2.docx
report-final2-new.docx

Confusing, right? Which one is the latest? Which one has the correct data? What if you want to **go back** to Day 1's version?

Version Control System (like Git) solves this problem!

- Git keeps track of every change (like a time machine).
- You can go back to *any previous version* easily.
- You don't need to create multiple final-final.docx files anymore!

Definition:
A **Version Control System (VCS)** is a tool that **records changes to files over time** so that you can recall specific versions later.

🔹 **What is Git?**

Think of **Git as a CCTV Camera for your code.**

- It watches every change (who did it, when, what was changed).
- It stores snapshots (commits) of your project.
- You can rewind and see the history.
- Multiple developers can work together and Git will track "who changed what".

🔹 In technical words:
Git is a **distributed version control system**. It runs on your **local machine** and manages code versions, branches, and merging.

- **Git** = Version Control System (VCS).
- Tracks project changes → allows rollback.
- Enables collaboration among developers.

- **What is GitHub?**

If **Git is your CCTV camera at home**, then **GitHub is your cloud storage (like Google Drive) where you upload those CCTV recordings**.

- GitHub is an **online platform** where you can store your Git repositories.
- A **cloud-based platform** hosting Git repositories.
- It helps you **share code with teammates**, manage collaboration, and contribute to open-source.
- Provides features like:
  - **Pull requests** (suggesting changes)
  - **Issues** (tracking bugs/tasks)
  - **Forks** (copy of someone else's repo)
  - **Branches** (parallel development)

- Think of Git as **your notebook** and GitHub as **Google Drive for your notebook**.
- **Git vs GitHub – Relation & Difference**

Git	GitHub
Tool to track changes (VCS).	Platform to host Git repositories.
Works on your **local system**.	Works **online (cloud)**.
Can be used alone (without GitHub).	Cannot work without Git.
Example: Track project locally.	Example: Share project with your team worldwide.

- Git = **Notebook** (where you write your code & track changes).
- GitHub = **Library** (where you store your notebook so that others can read, borrow, or contribute).

Why Use Both Git & GitHub Together?

- **Git alone** → You can track changes locally, but **can't collaborate easily**.
- **GitHub alone** → It's just a website, can't track versions without Git.
- **Together** → You get **version control + cloud hosting + collaboration tools**.

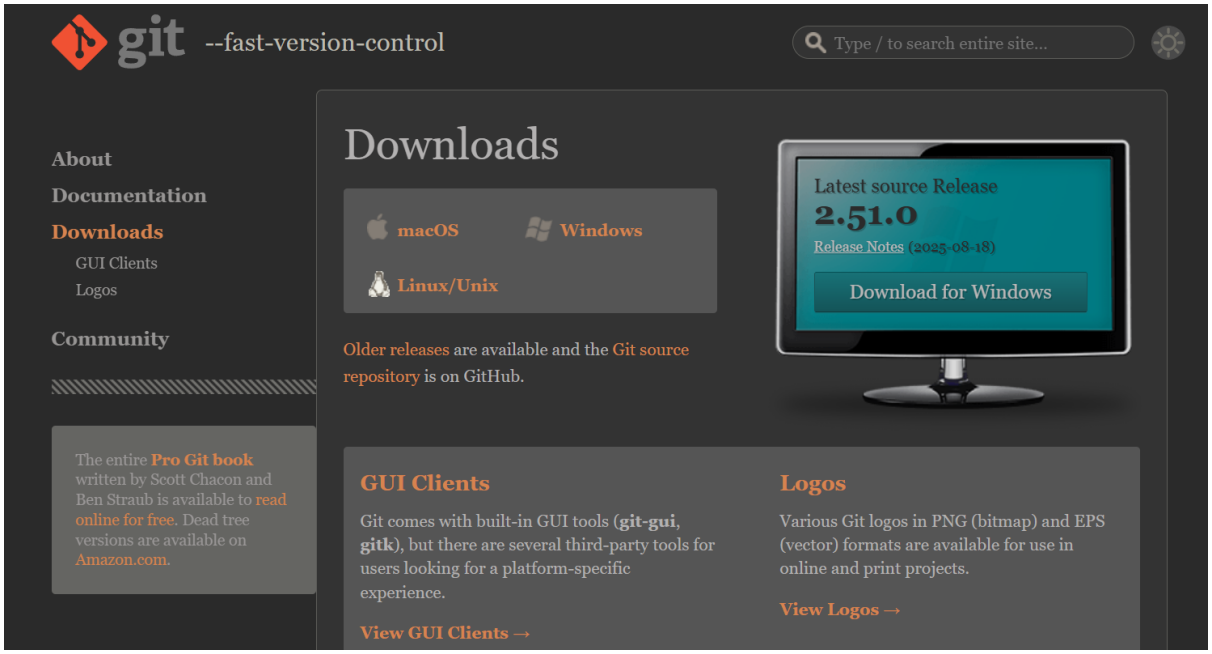
Example:
Imagine you and your friends are building a **mini web project**:

- On your laptop, Git tracks your changes (like saving checkpoints).
- You push it to GitHub → your friends can see and pull updates.
- They add new features → push again → you pull back.
- GitHub acts as a **shared code hub**, Git acts as the **history manager**.

1.7 Installation & Configuration of Git Account

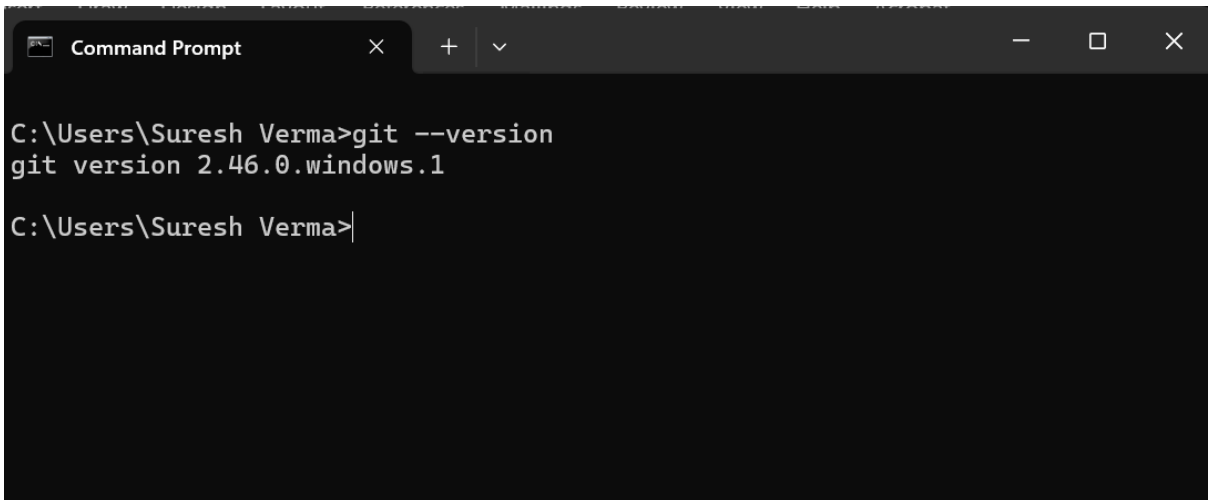
Steps:

1. Download: https://git-scm.com/downloads



2. Install with default settings.
3. Open terminal (CMD/VS Code terminal) → check version:

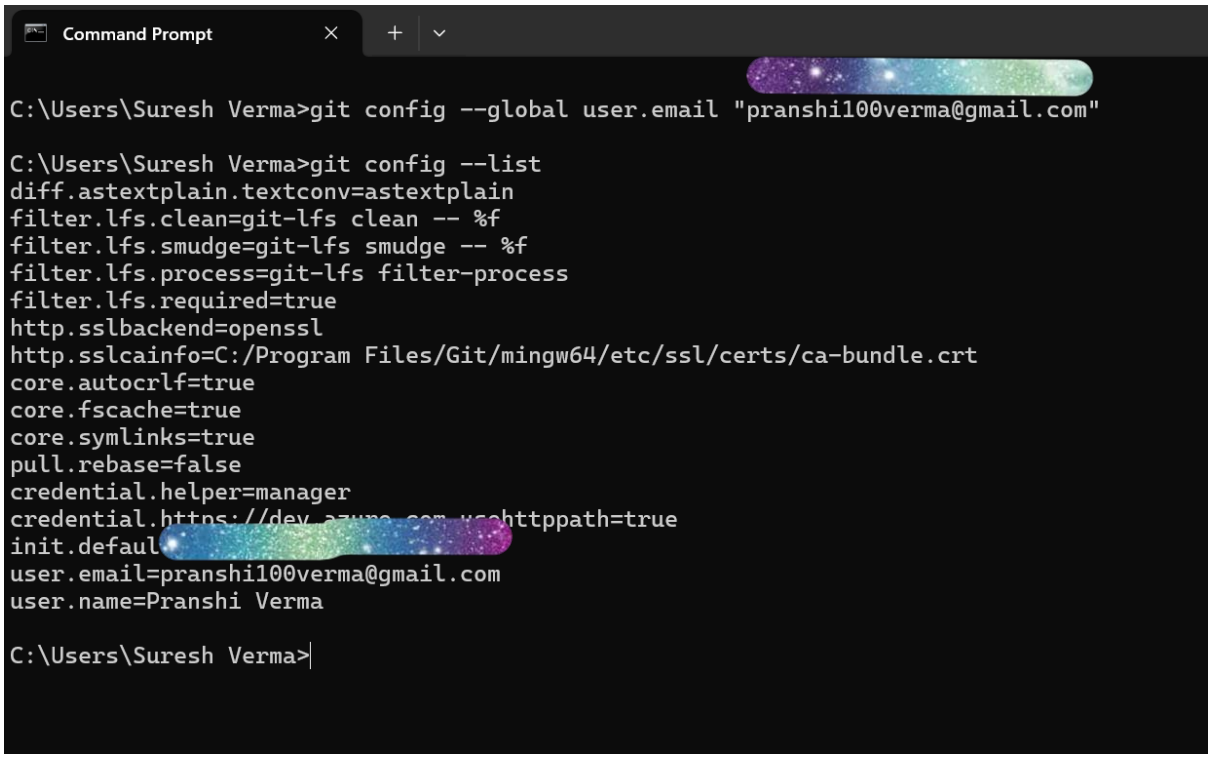
   git –version



Configure Git (first-time only):

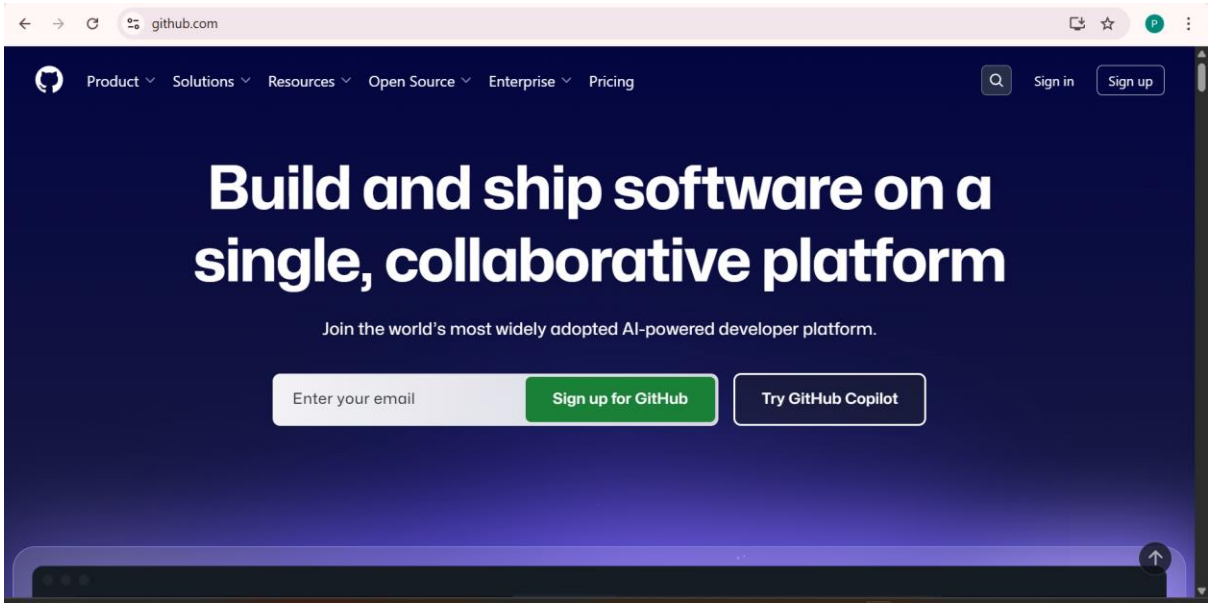
git config --global user.name "Your Name"
git config --global user.email "your_email@example.com"

git config --list # check configuration

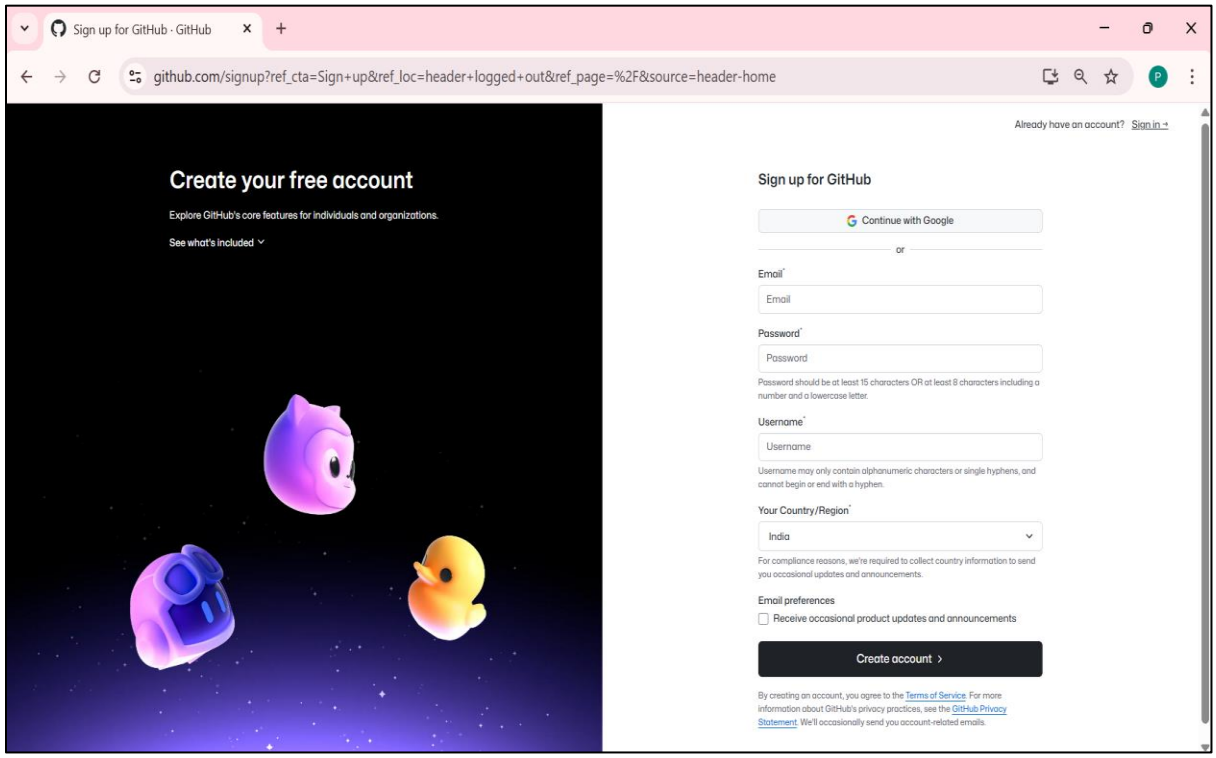


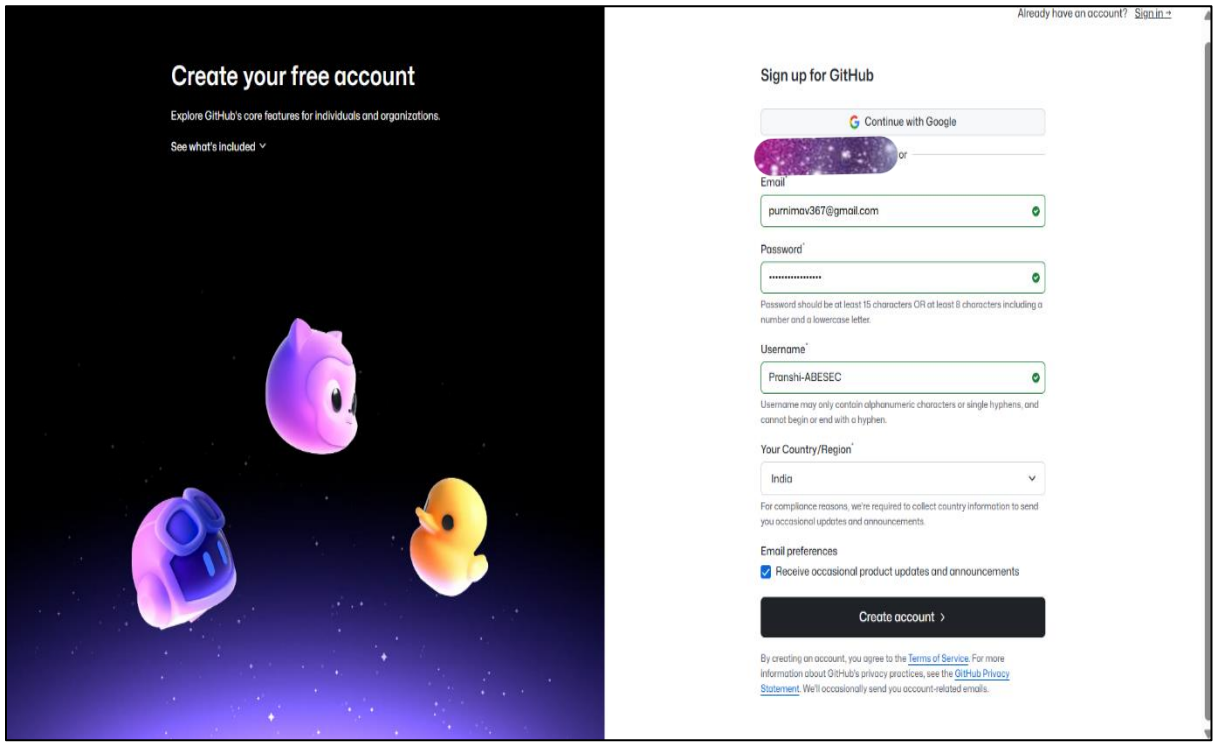
4. Creating a GitHub Account

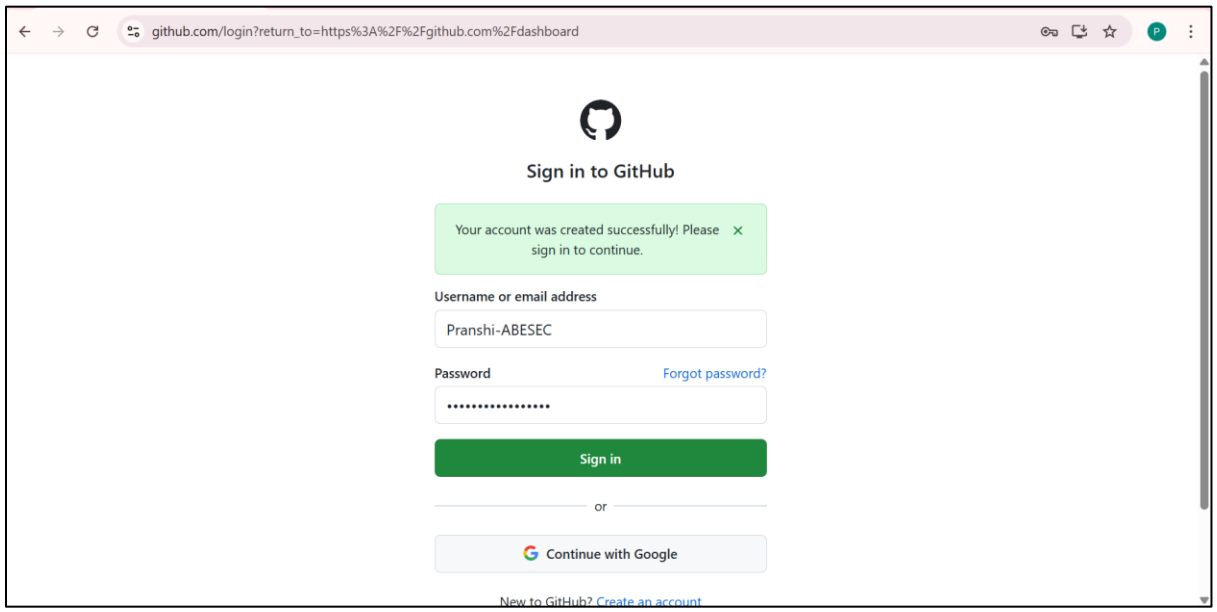
1. Go to https://github.com/



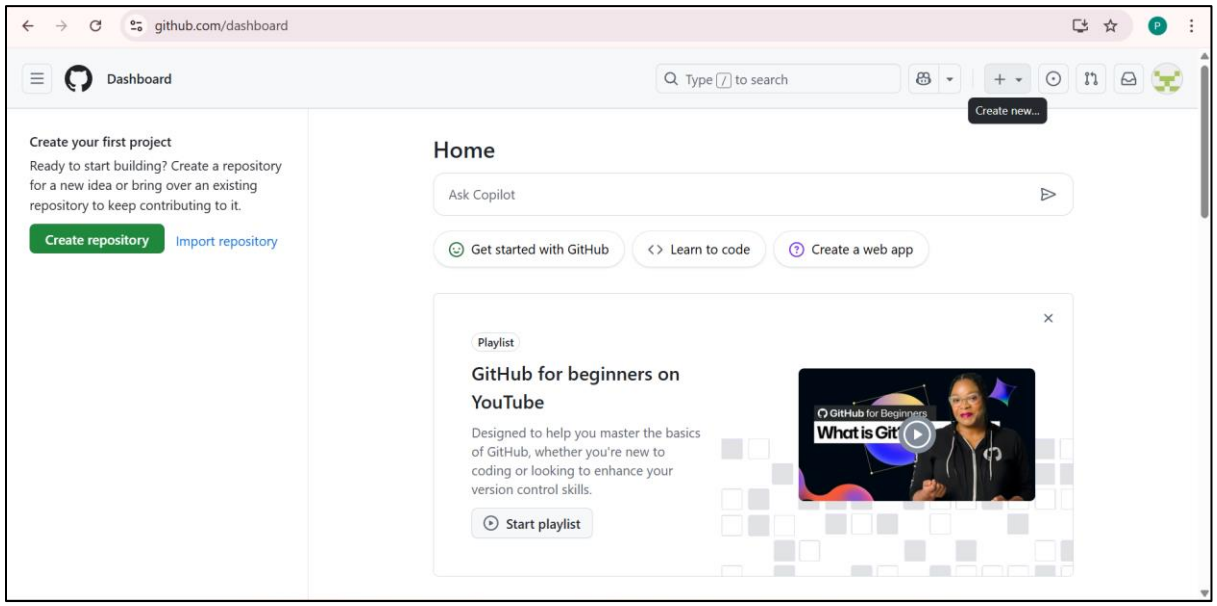
2. Sign up with email → verify → choose username.

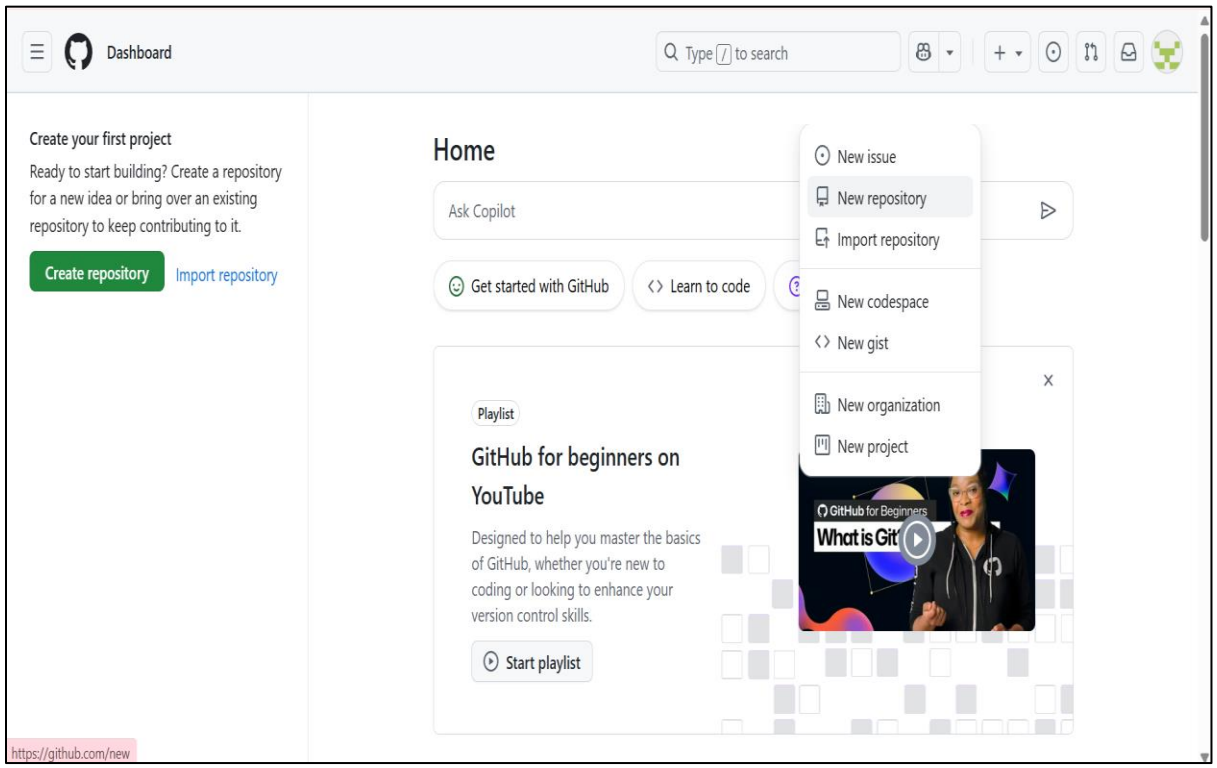

Sign up for GitHub · GitHub
github.com/signup?ref_cta=Sign+up&ref_loc=header+logged+out&ref_page=%2F&source=header-home
Already have an account? Sign in →
Create your free account
Explore GitHub's core features for individuals and organizations.
See what's included
Sign up for GitHub
Continue with Google
or
Email*
Email
Password*
Password
Password should be at least 15 characters OR at least 8 characters including a number and a lowercase letter.
Username*
Username
Username may only contain alphanumeric characters or single hyphens, and cannot begin or end with a hyphen.
Your Country/Region*
India
For compliance reasons, we're required to collect country information to send you occasional updates and announcements.
Email preferences
Receive occasional product updates and announcements
Create account ›
By creating an account, you agree to the Terms of Service. For more information about GitHub's privacy practices, see the GitHub Privacy Statement. We'll occasionally send you account-related emails.


Already have an account? Sign in →
Create your free account
Explore GitHub's core features for individuals and organizations.
See what's included
Sign up for GitHub
Continue with Google
or
Email
purnimav367@gmail.com
Password*
Password should be at least 15 characters OR at least 8 characters including a number and a lowercase letter.
Username*
Pranshi-ABESEC
Username may only contain alphanumeric characters or single hyphens, and cannot begin or end with a hyphen.
Your Country/Region*
India
For compliance reasons, we're required to collect country information to send you occasional updates and announcements.
Email preferences
Receive occasional product updates and announcements
Create account ›
By creating an account, you agree to the Terms of Service. For more information about GitHub's privacy practices, see the GitHub Privacy Statement. We'll occasionally send you account-related emails.

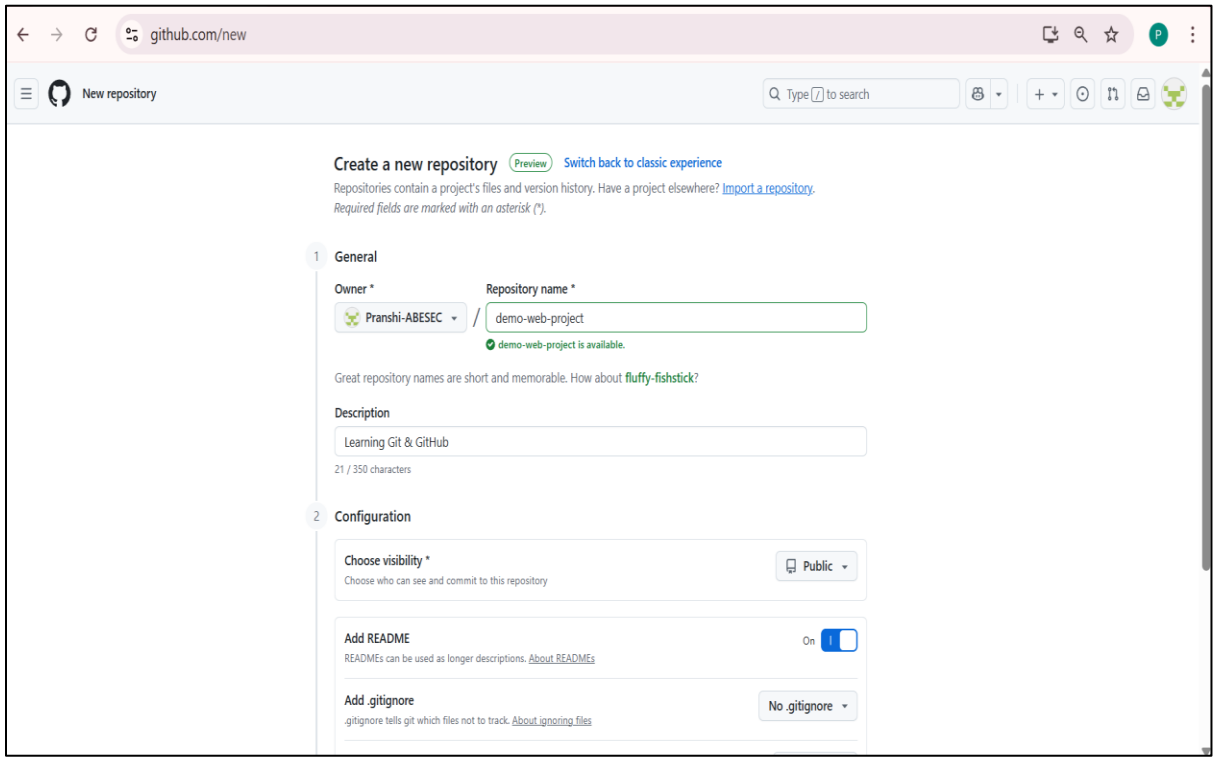


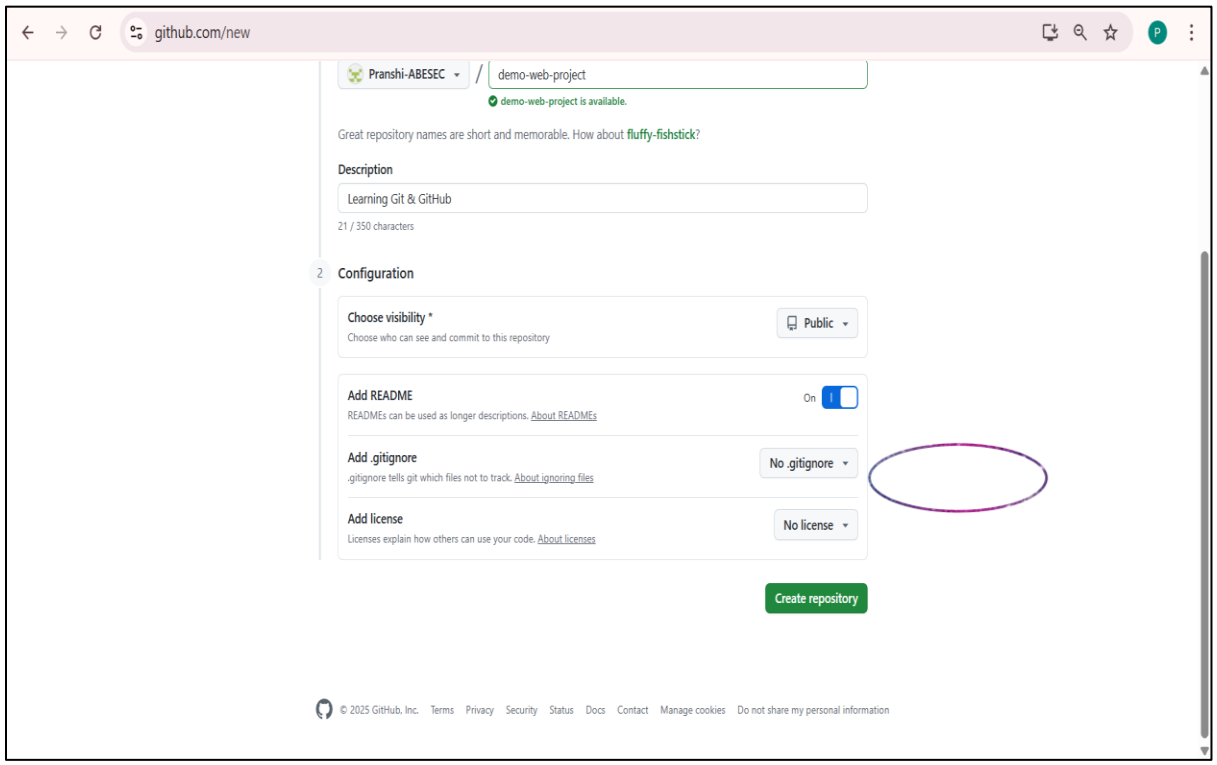
3. Create a new repository:





- Repository Name: demo-web-project
- Description: "Learning Git & GitHub"
- Public → Add README.md → Create Repository.





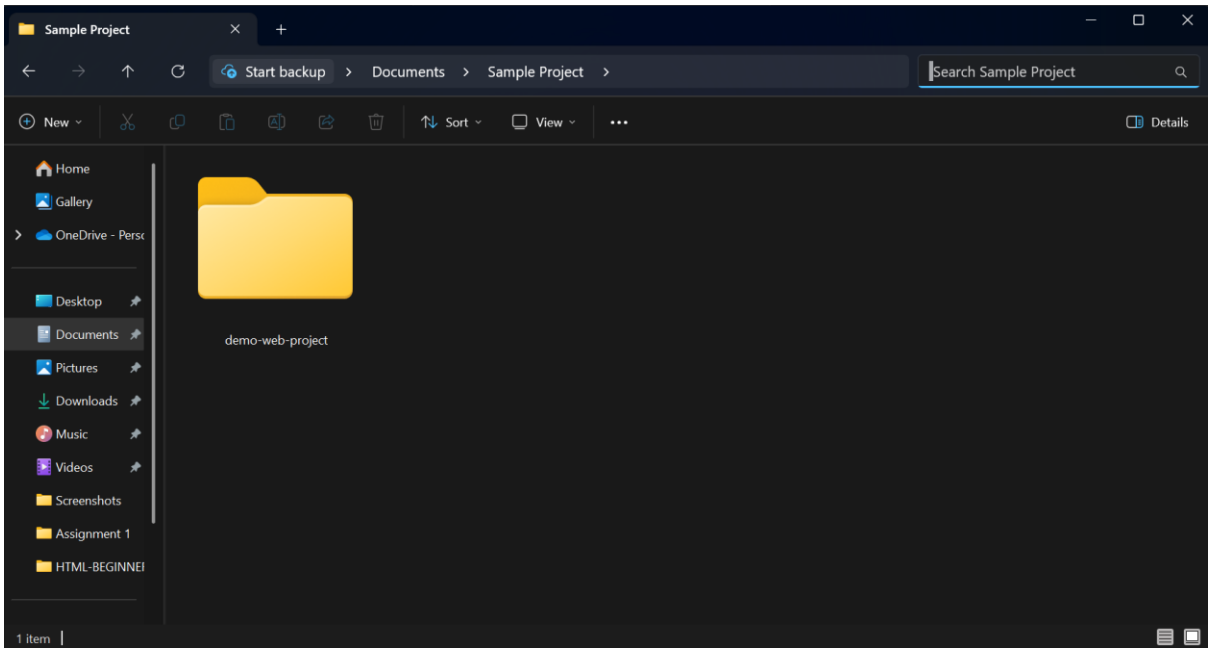
Creating a Local Repository

A **local repository** is a project folder on your computer that Git will track. This is the **first step** before pushing anything to GitHub.

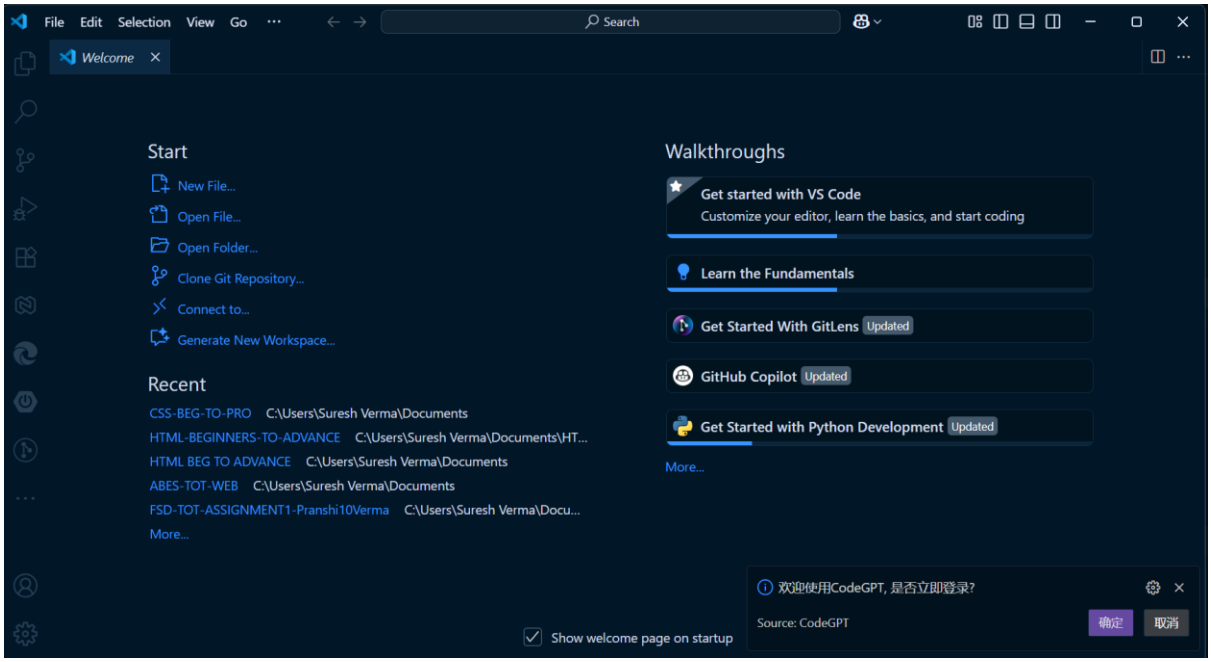
You can create it in two ways:

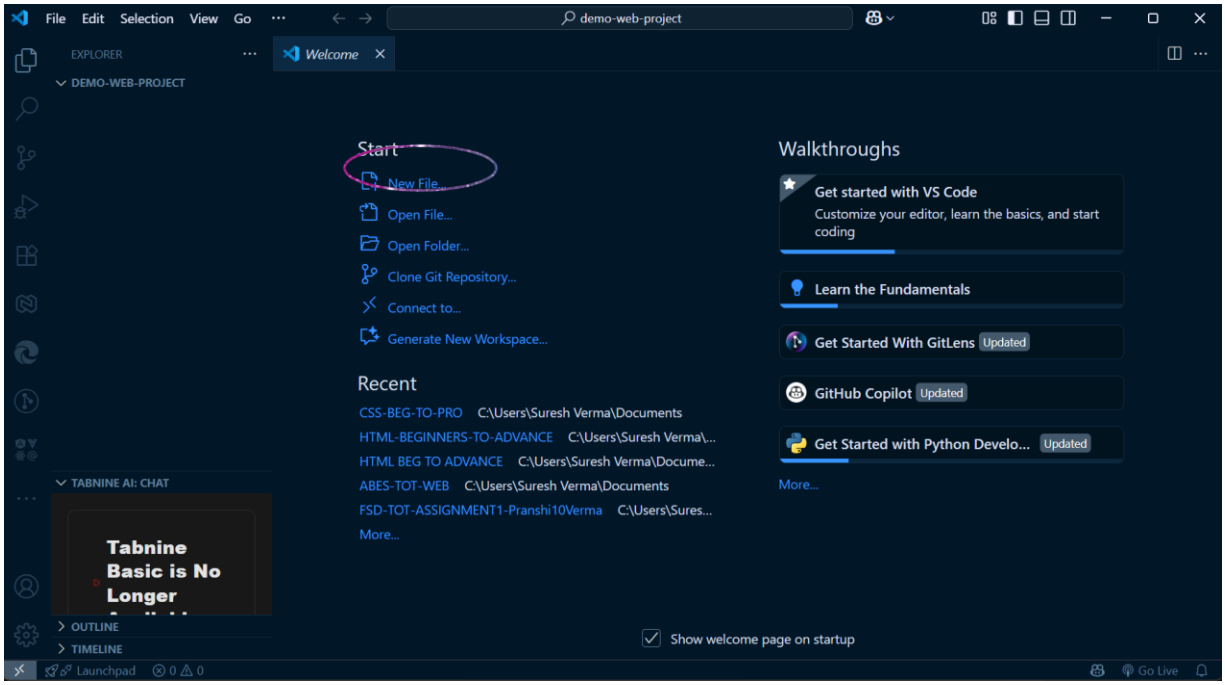
Method A – Using VS Code (Beginner-Friendly UI)

1. **Create a project folder**
    - Example: demo-web-project



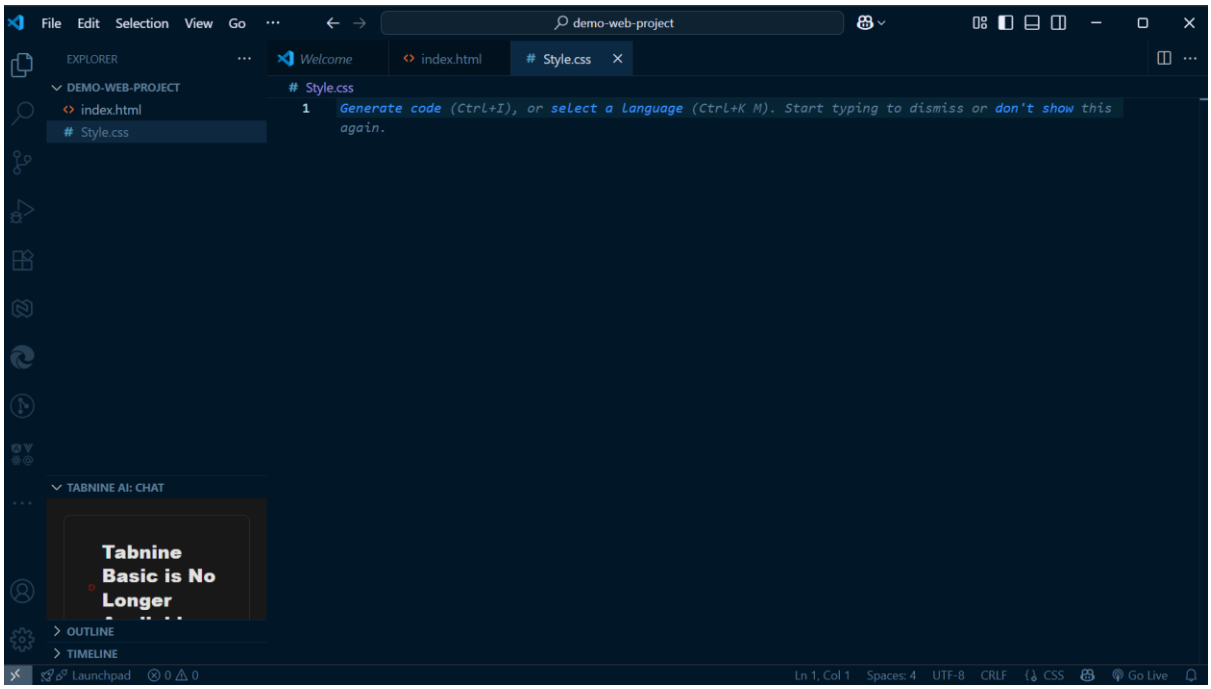
- Open it in VS Code (**File → Open Folder**).





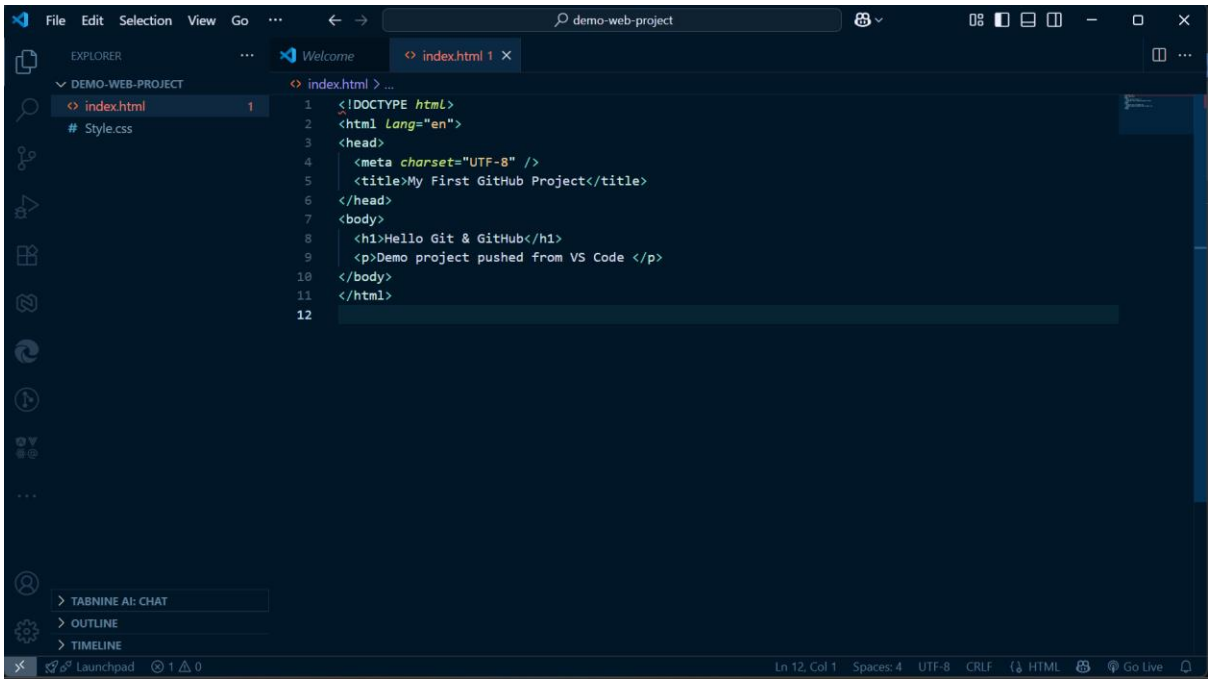
2. **Create project files**
   Example:

   - index.html
   - style.css



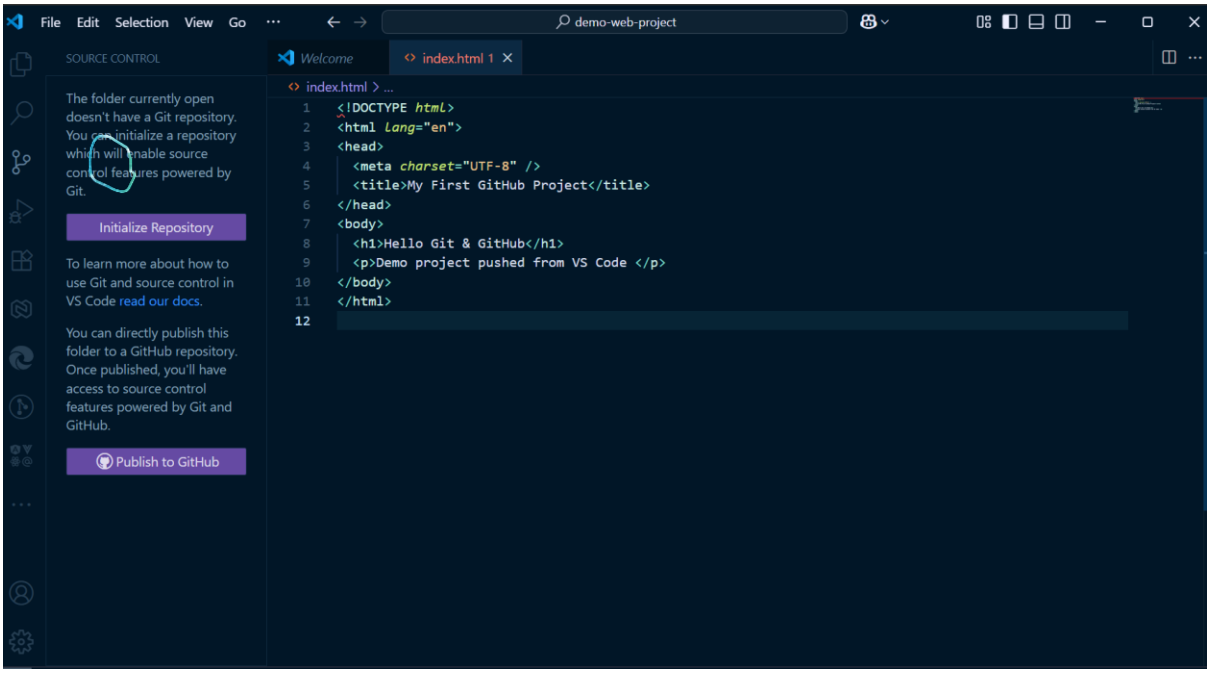
index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
 <meta charset="UTF-8" />
 <title>My First GitHub Project</title>
</head>
<body>
 <h1>Hello Git & GitHub</h1>
 <p>Demo project pushed from VS Code </p>
</body>
</html>
```



3. Initialize Git (create local repository)

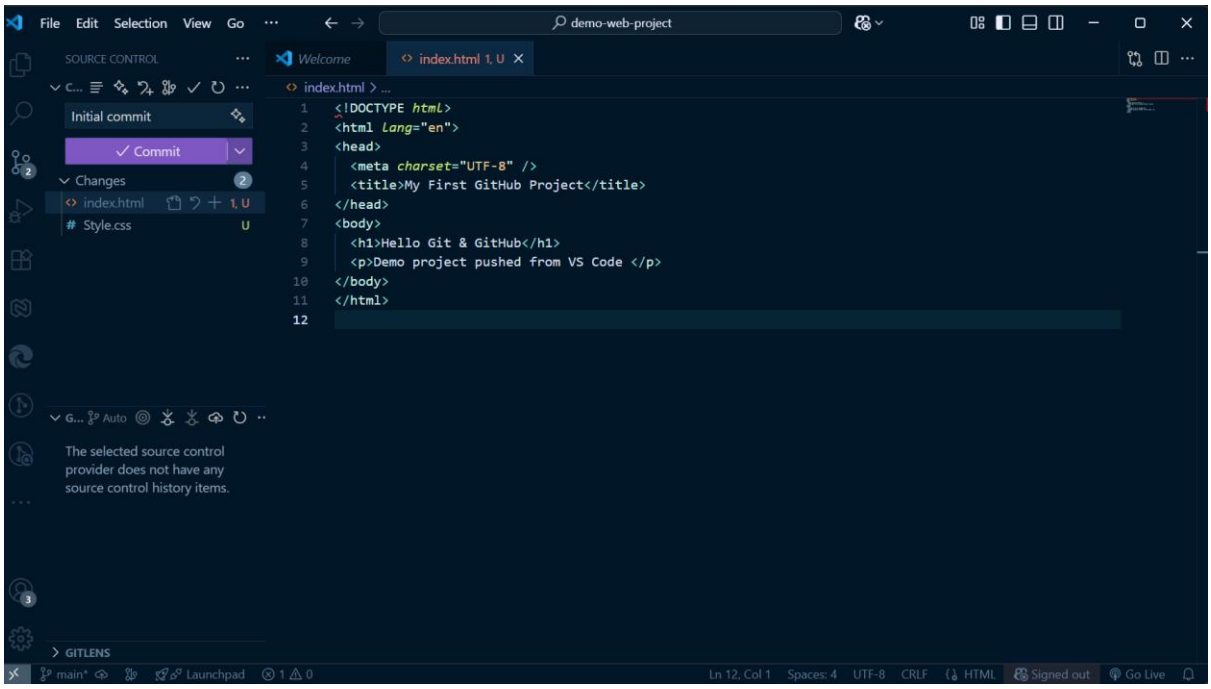
- Go to the **Source Control panel** in VS Code (branch icon on sidebar).



- Click **Initialize Repository**.
- This creates a **local Git repository** inside the folder.

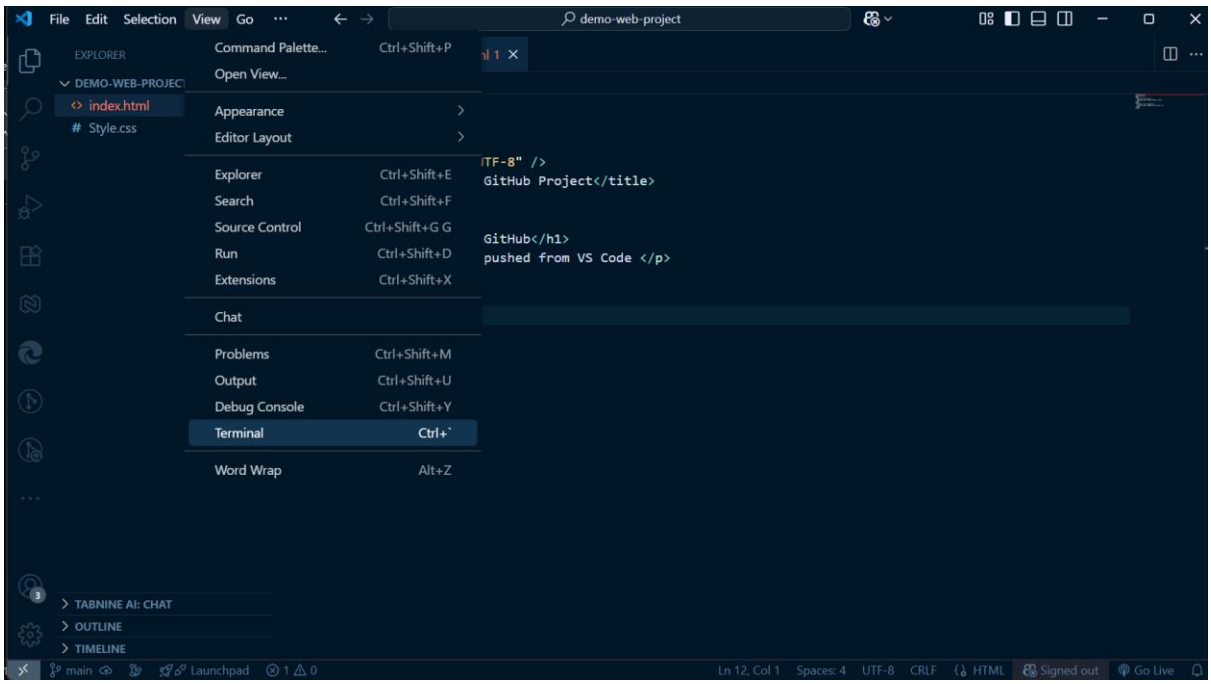
4. Make your first commit

- Stage changes (+ button).
- Enter a commit message: "Initial commit".
- Click **Commit**.



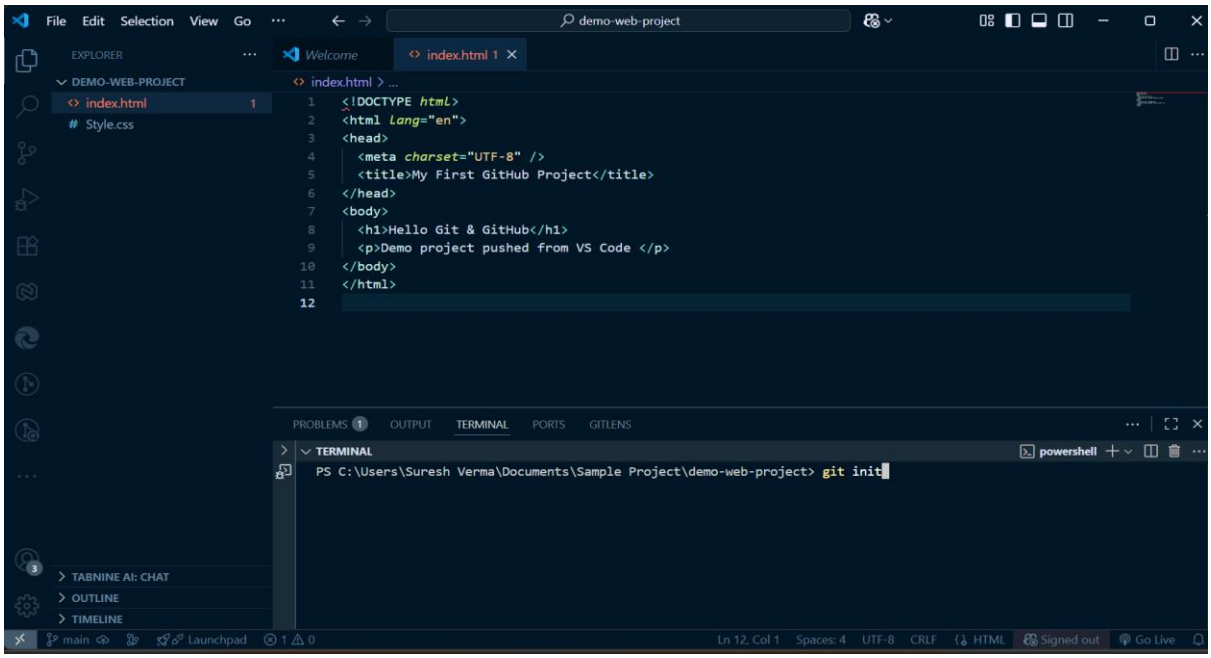
Method B – Using Terminal

1. Open **VS Code Terminal** (Ctrl + ~) or CMD.
2. Navigate to your project folder: *cd path/to/demo-web-project*



Initialize the repository:

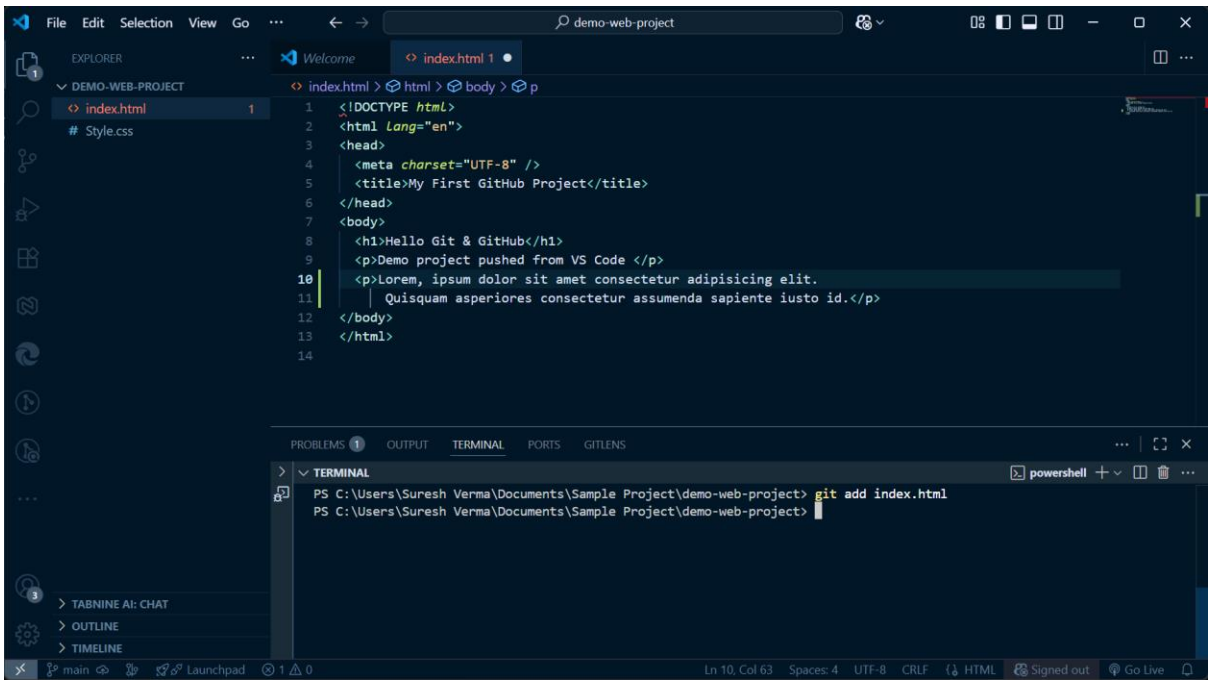
git init

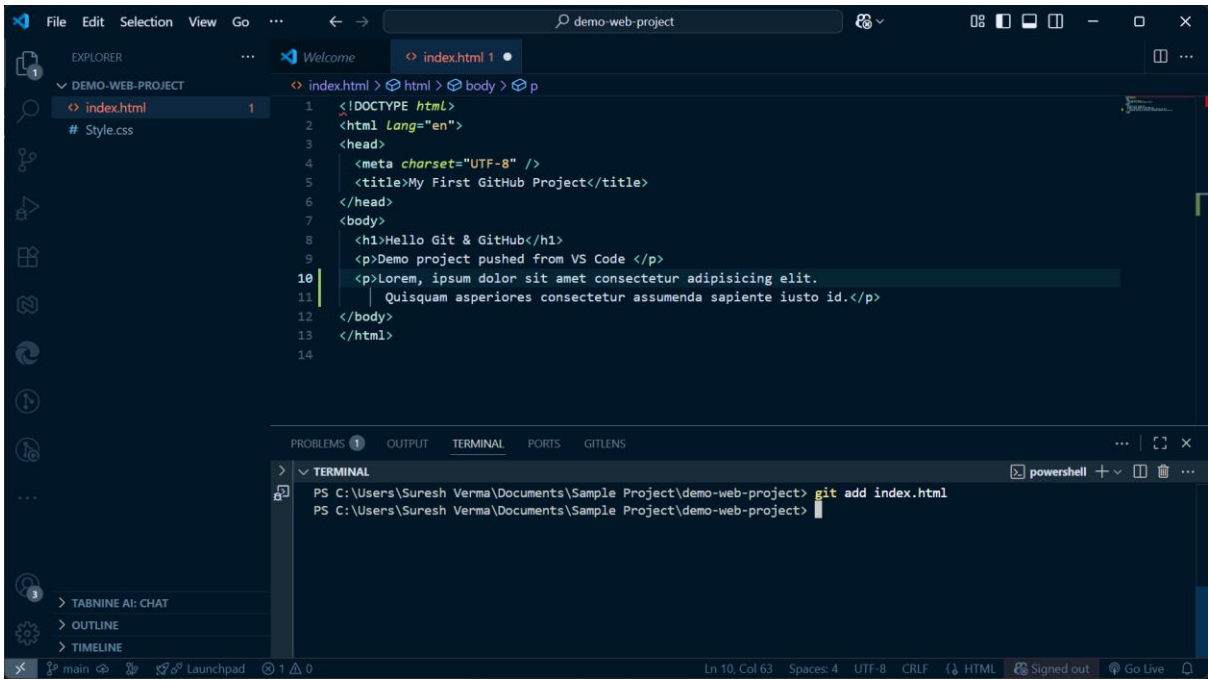


- This command creates a **hidden `.git` folder**, which means Git is now tracking your project.
- Add and commit files:

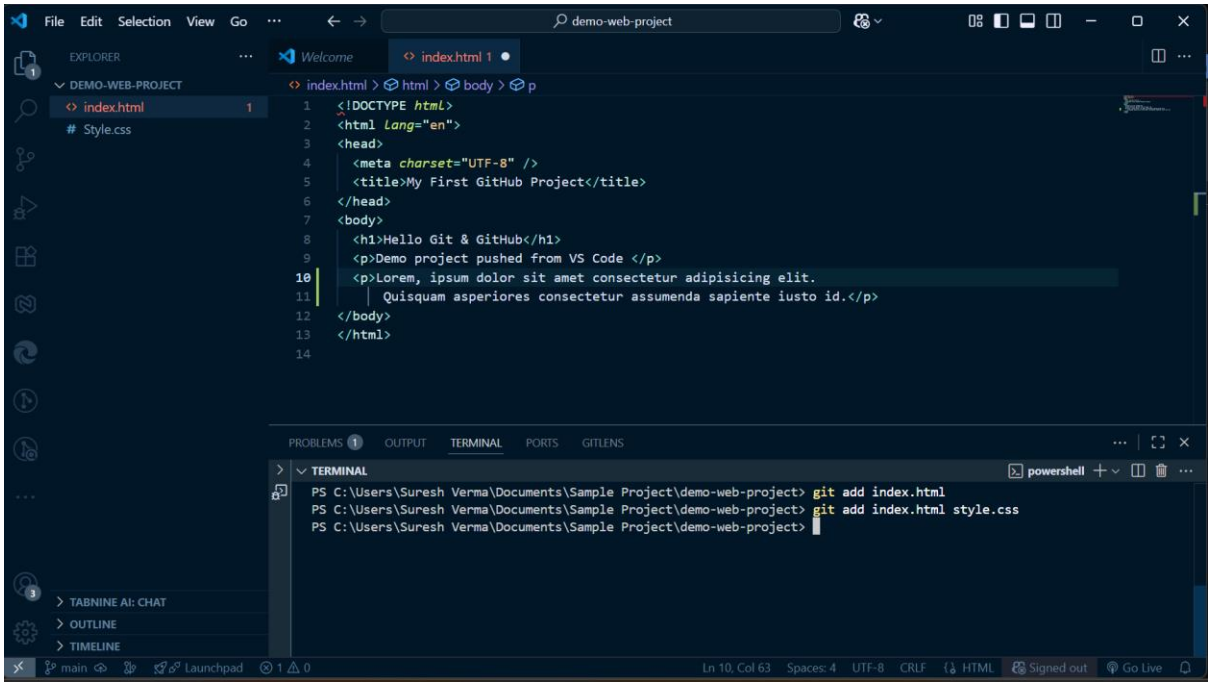
Add a Single File
If you only want to add **one file** to the staging area (for example, index.html): *git add index.html*

Add Multiple Specific Files
You can also add more than one file at a time by naming them: *Git add index.html*





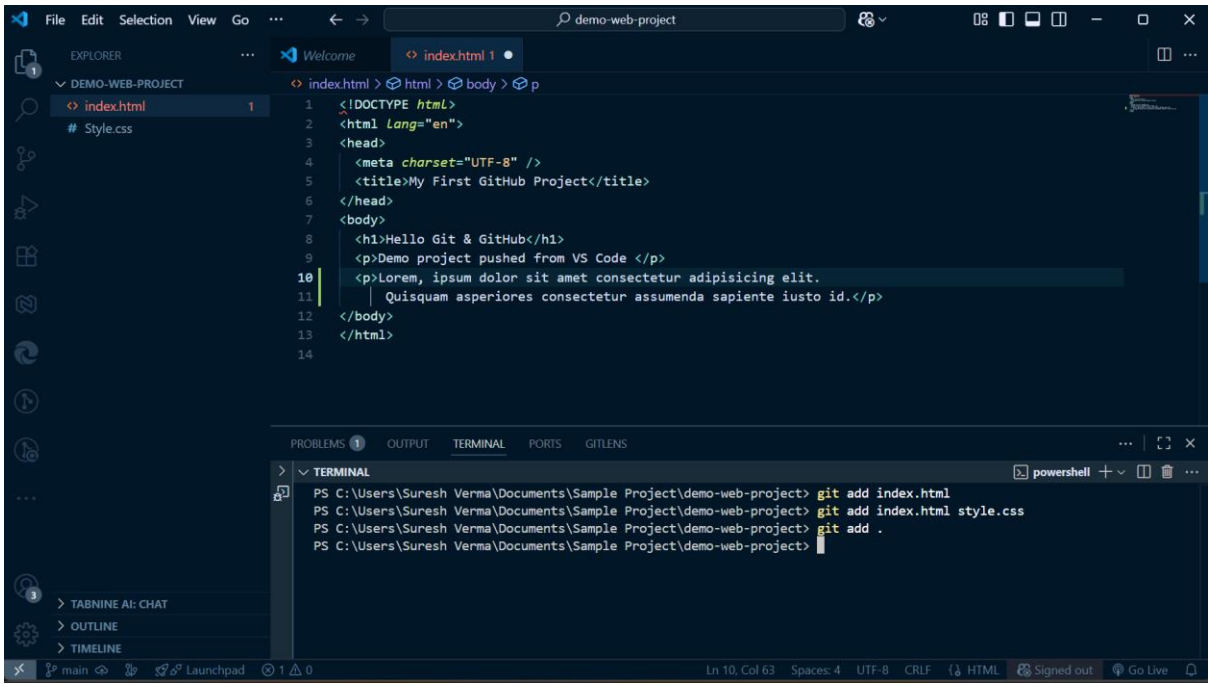
git add index.html style.css



Add All Files (Recommended when you want everything)

If you want to add **all files in the project folder**:

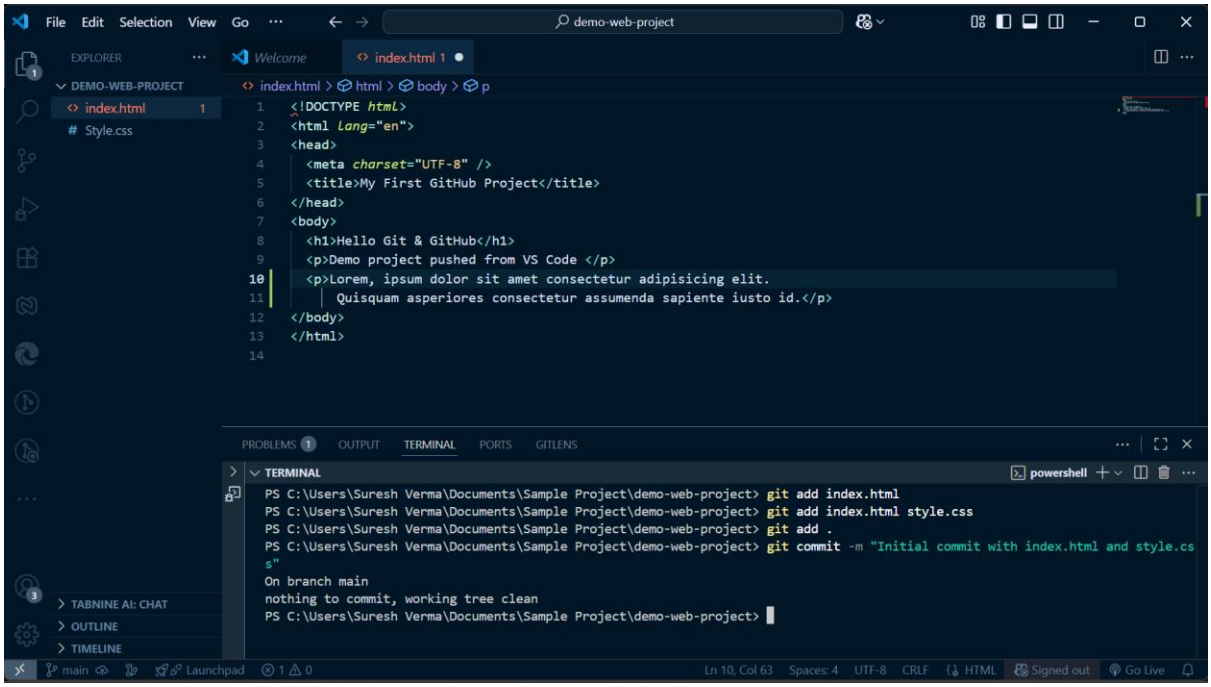
git add .



Commit the Changes

After adding files, you need to **commit** them with a message:

git commit -m "Initial commit with index.html and style.css"



A **commit** is like saving a snapshot of your project at that point in time.

Now we will link our local repository with the GitHub repository (remote).

Now your project is on GitHub!

Mr. Sanjeev – Notes + Practical (practicals are divided into sub-practicals (expandable format)

1.8 Introduction to HTML & HTML Features

1.9 HTML Page Structure

1.10 Elements in HTML (Semantic & Non-Semantic)

1.11 Types of Elements (Inline/Block)

1.12 Text Formatting: Headings, Paragraphs, Lists, Links, Images & Attributes

1.13 Comments in HTML, Formatting Elements, Special Characters & Symbols

1.14 Text-related Elements (Quotation & Citation)

1.15 Using Multimedia (Image/Video/Audio/YouTube) in HTML

1.16 Creating & Using Hyperlinks

1.8 Introduction to HTML & HTML Features:

What is HTML?

- HTML stands for Hyper Text Markup Language
- HTML is the standard markup language for creating Web pages
- HTML describes the structure of a Web page
- HTML consists of a series of elements
- HTML elements tell the browser how to display the content
- HTML elements label pieces of content such as "this is a heading", "this is a paragraph", "this is a link", etc.

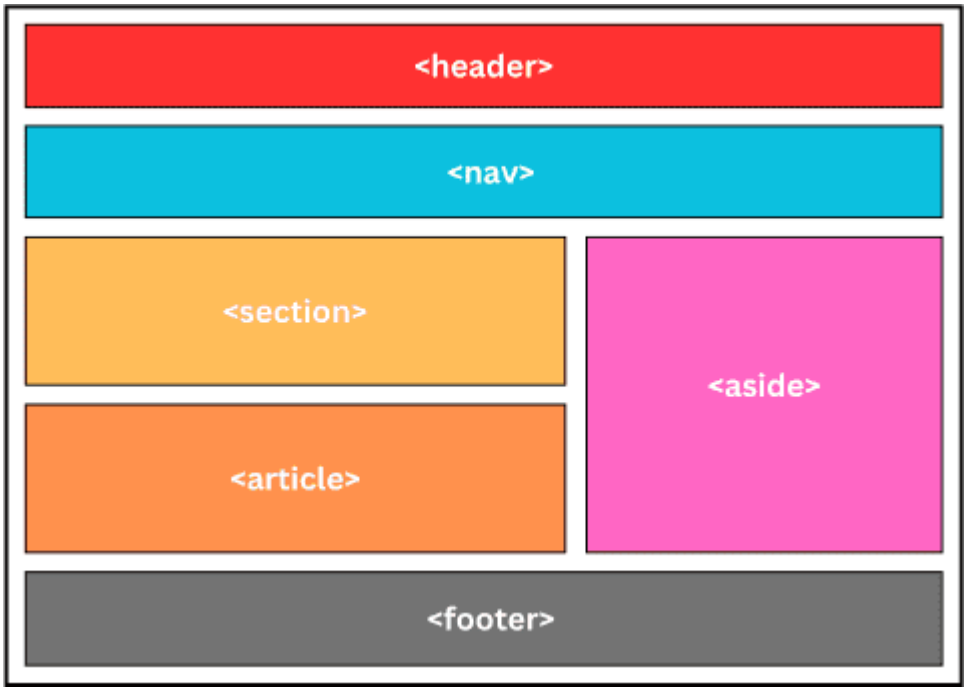
HTML 5 Features

HTML5 was introduced having some new features following are

- Error tolerance
- Save as html or .htm
- Pass data in "……." or '..... '
- Name Casing <HTML> or <html>
- Semantic Elements
- Audio and Video Support
- Canvas Elements
- Geolocation API
- Local Storage
- Responsive Images
- Web Workers
- Drag and Drop API
- Form Enhancements
- Web Sockets
- Micro Data

1.9 HTML Page Structure:

An HTML page is structured using a hierarchical arrangement of elements, providing a logical organization for the content. The fundamental structure includes:



<!DOCTYPE html> Declaration:

This declaration, placed at the very beginning of an HTML document, informs the browser that the document is an HTML5 document. It is not an HTML tag but a directive.

<html> Element:

This is the root element of an HTML page, encapsulating all other HTML elements. It signifies the beginning and end of the HTML document.

<head> Element:

Located within the <html> element, the <head> contains metadata about the HTML document. This information is not directly displayed on the web page but is crucial for browser rendering, search engine optimization, and other functionalities. Common elements within the <head> include:

<title>: Defines the title of the document, which appears in the browser tab or window title bar.

<meta>: Provides various metadata, such as character set (charset), viewport settings, and descriptions for search engines.

<link>: Links external resources like stylesheets (.css files) or favicons.

<style>: Embeds internal CSS styles directly within the HTML document.

<script>: Embeds or links external JavaScript code for interactive functionality.

<body> Element:

Also located within the <html> element, the <body> contains all the visible content of the web page. This is where the actual content that users interact with is placed. Examples of elements found within the <body> include:

Headings (<h1> to <h6>): Define titles and subtitles for sections of content.

Paragraphs (<p>): Enclose blocks of text.

Links (<a>): Create hyperlinks to other web pages or resources.

Images (<img>): Embed images into the page.

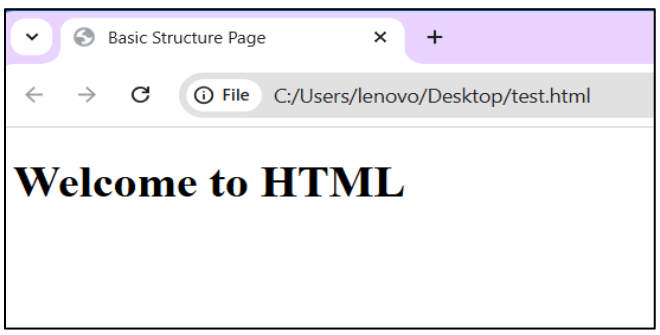
Lists (<ul>, <ol>, <li>): Create unordered or ordered lists of items.

Divisions ( ): Generic container for grouping other elements for styling or scripting purposes. Provide more meaningful structure to the content, improving accessibility and SEO.

This nested structure, starting from the <!DOCTYPE html> and encompassing the <html>, <head>, and
<body> elements, forms the fundamental framework of every HTML page.

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport" content="width=device-width, initial-scale=1.0">
   <title>Basic Structure Page</title>
</head>
<body>
   <h1>Welcome to HTML</h1>
</body>
</html>
```

Output:



1.10 Elements in HTML(Semantic and Non Semantic)

A tag is a keyword that defines how a web browser should format and display a web page. HTML tags are the basic building blocks of any website. In html, codes(normal text) enclosed in brackets written in usually paired.

See below example:

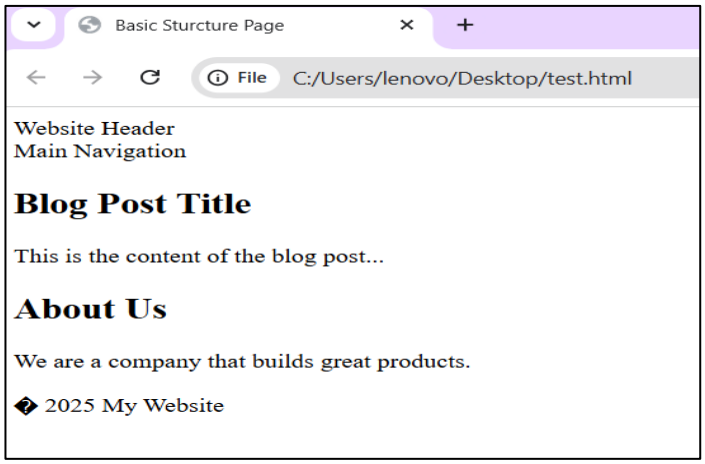
<TITLE>My Web Page</TITLE>

Note: - HTML is not case sensitive. <TITLE> = <title> =

<TItLE> In HTML,

```
<!DOCTYPE html>
<html lang="en">
 <head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Basic Sturcture Page</title>
 </head>
 <body>
  <header>Website Header</header>
  <nav>Main Navigation</nav>
  <main>
   <article>
    <h2>Blog Post Title</h2>
    <p>This is the content of the blog post...</p>
   </article>
   <section>
    <h2>About Us</h2>
    <p>We are a company that builds great products.</p>
   </section>
  </main>
  <footer>© 2025 My Website</footer>
 </body>
```

Output:



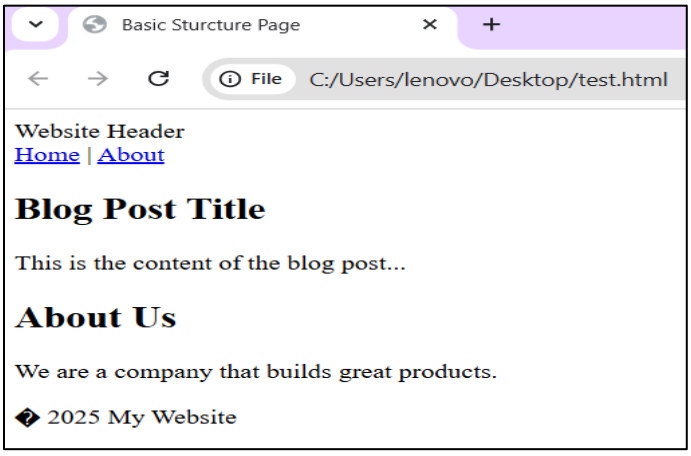
Semantic tags define the meaning of the content they contain, while non-semantic tags only old content and do not indicate its role on the page.
Semantic tags are both human- and machine-readable, and they can help users and machines understand the context and importance of the content.

Non-semantic tags, on the other hand, are used for styling purposes or as containers for other elements.

For Example: the tag does not suggest the content it will carry, however using the <p> tag suggests it can be used to hold paragraph information.

```
<!DOCTYPE html>
<html lang="en">
 <head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Basic Sturcture Page</title>
 </head>
 <body>
  <div id="header">Website Header</div>
  <div id="menu"><a href="#">Home</a> | <a href="#">About</a></div>
  <div id="content">
   <div class="post">
    <h2>Blog Post Title</h2>
    <p>This is the content of the blog post...</p>
   </div>
   <div class="info">
    <h2>About Us</h2>
    <p>We are a company that builds great products.</p>
   </div>
  </div>
  <div id="footer">© 2025 My Website</div>
 </body>
```

Output:



There are many reasons why you should write Semantic tags instead of normal HTML tags, to leverage SEO, accessibility, and browser compatibility.

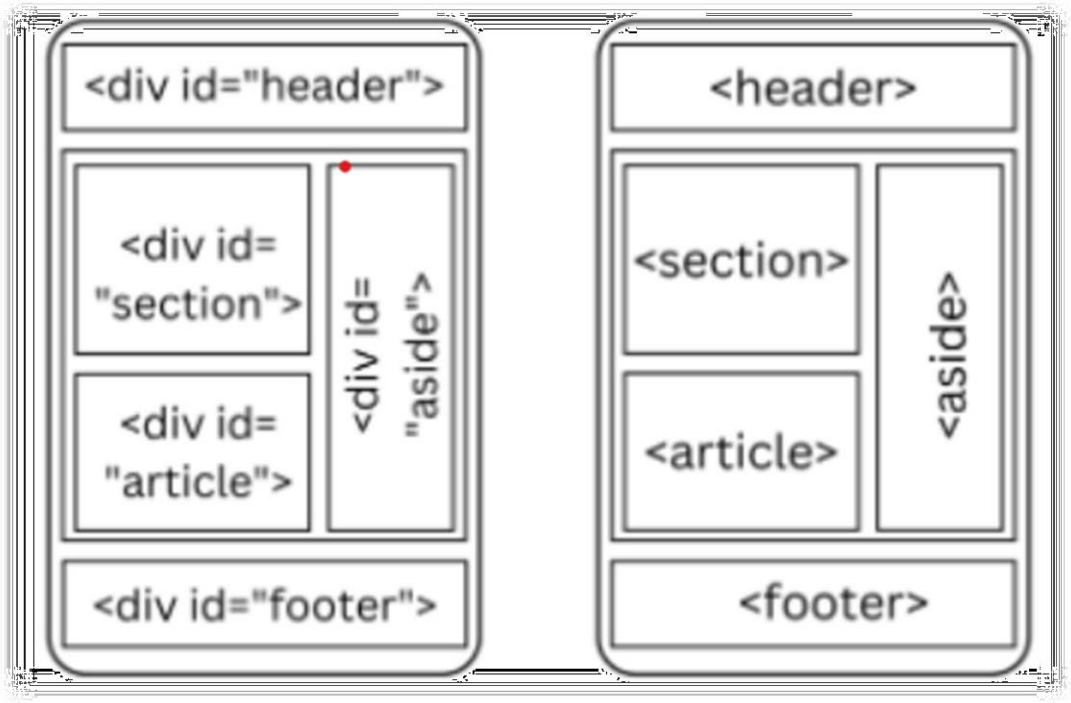


Fig: Non-Semantic and Semantic Tags

1.11 Types of Elements (Inline/Block)

HTML elements can be categorized in several ways, primarily based on their display behavior and their purpose.

1. Block-level Elements:
These elements typically start on a new line and take up the full available width of their parent container.
They are used to structure the main sections of a web page.
Examples include:
<address>,<article>,<aside>,<blockquote>,<canvas>,<dd>, ,<dl>,<dt>,<fieldset>, <figcaption>,<figure>,<footer>,<form>,<h1>…<h6>,<header>,<hr>,<li>,<main>,<nav>,<noscript>, <ol>,<p>,<pre>,<section>,<table>,<tfoot>,<ul>,<video>.

2. Inline Elements:
These elements do not start on a new line and only take up as much width as their content requires. They are used to style or structure content within a block-level element.
Examples include:
<a>,<abbr>,<acronym>,<b>,<bdo>,<big>,
,<button>,<cite>,<code>,<dfn>,,<i>,<img>,<input>,

<kbd>,<label>,<map>,<object>,<output>,<q>,<samp>,<script>,<select>,<small>,,,<sub>,<sup>,<textarea>,<time>,<tt>,<var>.

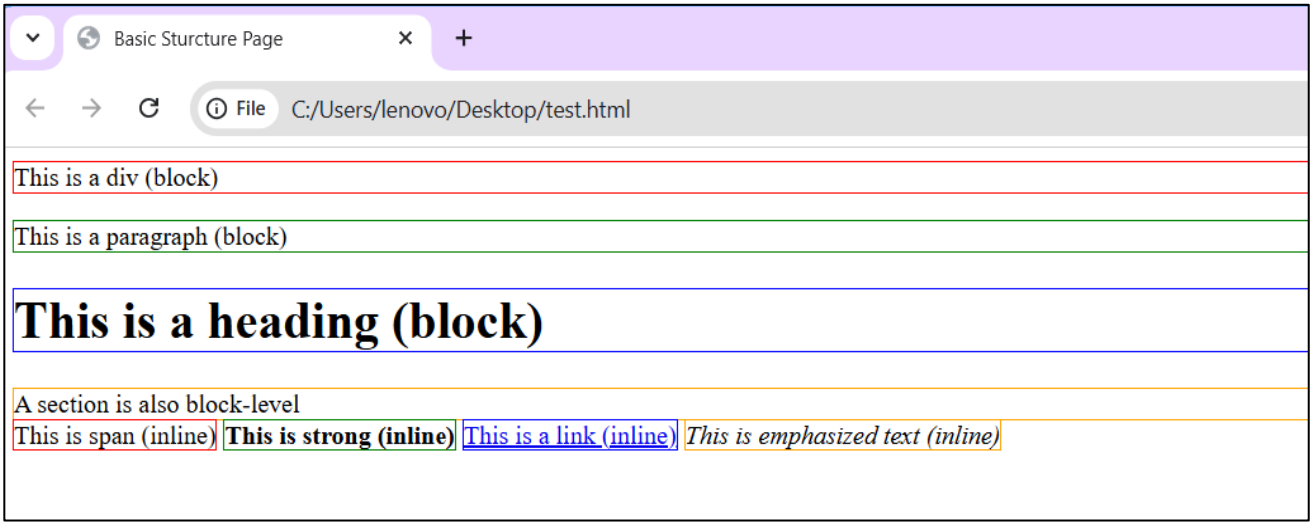
```
<!DOCTYPE html>
<html lang="en">
 <head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Basic Sturcture Page</title>
 </head>
 <body>
  <!--block level elements-->
  <div style="border: 1px solid red">This is a div (block)</div>

  <p style="border: 1px solid green">This is a paragraph (block)</p>

  <h1 style="border: 1px solid blue">This is a heading (block)</h1>

  <section style="border: 1px solid
   orange"> A section is also block-level
  </section>
  <!--inline elements-->
  <span style="border: 1px solid red">This is span (inline)</span>
  <strong style="border: 1px solid green">This is strong (inline)</strong>
  <a href="#" style="border: 1px solid blue">This is a link (inline)</a>
  <em style="border: 1px solid orange">This is emphasized text (inline)</em>
 </body>
```

Output:



Tag	Description
<html> ... </html>	Declares the Web page to be written in HTML
<head> ... </head>	Delimits the page's head
<title> ... </title>	Defines the title (not displayed on the page)
<body> ... </body>	Delimits the page's body
<h *n*> ... </h*n*>	Delimits a level *n* heading
...	Set ... in boldface
...	Set ... in italics
...	Center ... on the page horizontally
...	Brackets an unordered (bulleted) list
...	Brackets a numbered list
...	Brackets an item in an ordered or numbered list
 	Forces a line break here
<p>	Starts a paragraph
<hr>	Inserts a horizontal rule
<img src="...">	Displays an image here
 ... 	Defines a hyperlink

1.13 HTML Comments

```
<!DOCTYPE html>

<html lang="en">

 <head>

  <meta charset="UTF-8" />

  <meta name="viewport" content="width=device-width, initial-scale=1.0" />

  <title>HTML Comments</title>

 </head>

 <body> <!—body start -->

<!—section is started from here! -->

<section>

<h2> HTML Comments</h2>
<p>The comment tag is used to insert comments in the source code. Comments are not displayed in the browsers.<br> You can use comments to explain your code, which can help you when you edit the source code at a later date. <br>This is especially useful if you have a lot of code..</p>

 </section>

</body> <!--body end--></html>
-------------------------
```

Comments can be inserted into the HTML code to make it more readable and understandable. Comments are ignored by the browser and are not displayed.

Output:

1.12 Text Formatting Elements:

In this chapter, you will learn how to enhance your page with Bold, Italics, and other character formatting options.

Objectives: Upon completing this section, you should be able to change the color and size of your text. Where we use Common Character Formatting Elements even can add special characters.

<FONT SIZE="+2"> Two sizes bigger</FONT>

The size attribute can be set as an absolute value from 1 to 7 or as a relative value using the "+" or "-" sign. Normal text size is 3 (from -2 to +4).

<FONT COLOR="#RRGGBB">this text has color</FONT>

Color = "#RRGGBB" The COLOR attribute of the FONT element.

<B> Bold </B>

<I> Italic </I>

<U> Underline </U>

<PRE> Preformatted </PRE>

Text enclosed by PRE tags is displayed in a mono-spaced font. Spaces and line breaks are supported without additional elements or special characters.

<EM> Emphasis </EM>

Browsers usually display this as italics.

<STRONG> STRONG </STRONG>

Browsers display this as bold.

<TT> TELETYPE </TT>

Text is displayed in a mono-spaced font. A typewriter text, e.g. fixed-width font.

<STRIKE> strike-through text</STRIKE>

DEL is used for STRIKE at the latest browsers

<BIG> places text in a big font</BIG>

<SMALL> places text in a small font</SMALL>

_{places text in subscript position}

^{places text in superscript style position}

<ins>showing new inserted text</ins>

<mark>used for

highlighting</mark> HTML

ENTITIES

Some characters are reserved in HTML.

If you use the less than (<) or greater than (>) signs in your HTML text, the browser might mix them with tags.

- Entity names or entity numbers can be used to display reserved HTML characters.
- Entity names look like this: &entity_name;
- Entity numbers look like this: &#entity_number;

HTML ENTITIES or Characters & Symbols

These Characters are recognized in HTML as they begin with an ampersand and end with with a semi- colon e.g. &value; The value will either be an entity name or a standard ASCII character number. They are called escape sequences.

The next table represents some of the more commonly used special characters. For a comprehensive listing.

ASSIGNMENT

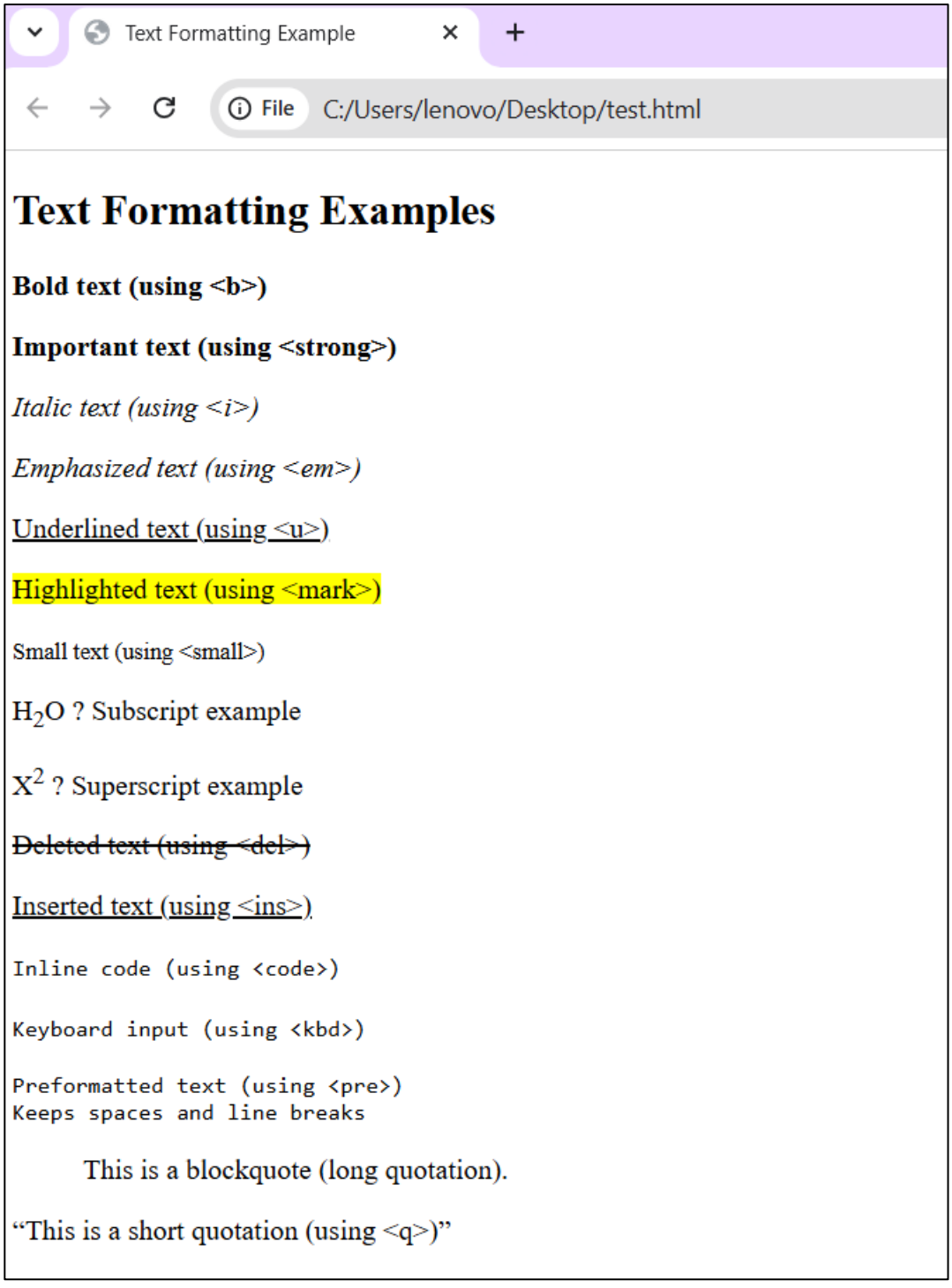
- **Design a basic HMTL page using any two semantic and non-semantic elements.**
- **Design a HTML page creating hyperlink for text, images and all media elements.**

```
!DOCTYPE html>
html lang="en">
head>
<meta charset="UTF-8">
<title>Text Formatting Example</title>
/head>
body>
<h2>Text Formatting Examples</h2> <p><b>Bold text (using
&lt;b&gt;)</b></p> <p><strong>Important text (using
&lt;strong&gt;)</strong></p> <p><i>Italic text (using
&lt;i&gt;)</i></p> <p><em>Emphasized text (using
&lt;em&gt;)</em></p> <p><u>Underlined text (using
&lt;u&gt;)</u></p> <p><mark>Highlighted text (using
&lt;mark&gt;)</mark></p> <p><small>Small text (using
&lt;small&gt;)</small></p> <p>H<sub>2</sub>O → Subscript
xample</p> <p>X<sup>2</sup> → Superscript example</p>
<p><del>Deleted text (using &lt;del&gt;)</del></p>
<p><ins>Inserted text (using &lt;ins&gt;)</ins></p>
<p><code>Inline code (using &lt;code&gt;)</code></p>
<p><kbd>Keyboard input (using &lt;kbd&gt;)</kbd></p>
<p><pre>

reformatted text (using &lt;pre&gt;)
 Keeps spaces and line breaks
</pre></p>
<p><blockquote cite="https://www.example.com">
 This is a blockquote (long quotation).
</blockquote></p>
<p><q>This is a short quotation (using
lt;q&gt;)</q></p> </body>

/html>
```

Output:


Text Formatting Example
File
C:/Users/lenovo/Desktop/test.html
Text Formatting Examples
Bold text (using <b>)
Important text (using)
Italic text (using <i>)
Emphasized text (using)
Underlined text (using <u>)
Highlighted text (using <mark>)
Small text (using <small>)
H2O ? Subscript example
X2 ? Superscript example
Deleted text (using <del>)
Inserted text (using <ins>)
Inline code (using <code>)
Keyboard input (using <kbd>)
Preformatted text (using <pre>)
Keeps spaces and line breaks
This is a blockquote (long quotation).
“This is a short quotation (using <q>)”

Special Characters & Symbols OR ENTITIES

Special Character	Entity Name	Special Character	Entity Name
Ampersand	& &	Greater-than sign	> >
Asterisk	∗ ∗∗	Less-than sign	< <
Cent sign	¢ ¢	Non-breaking space	
Copyright	© ©	Quotation mark	" "
Fraction one qtr	¼ ¼	Registration mark	® ®
Fraction one half	½ ½	Trademark sign	™ ™

HTML ENTITIES or Characters & Symbols

Additional escape sequences support accented

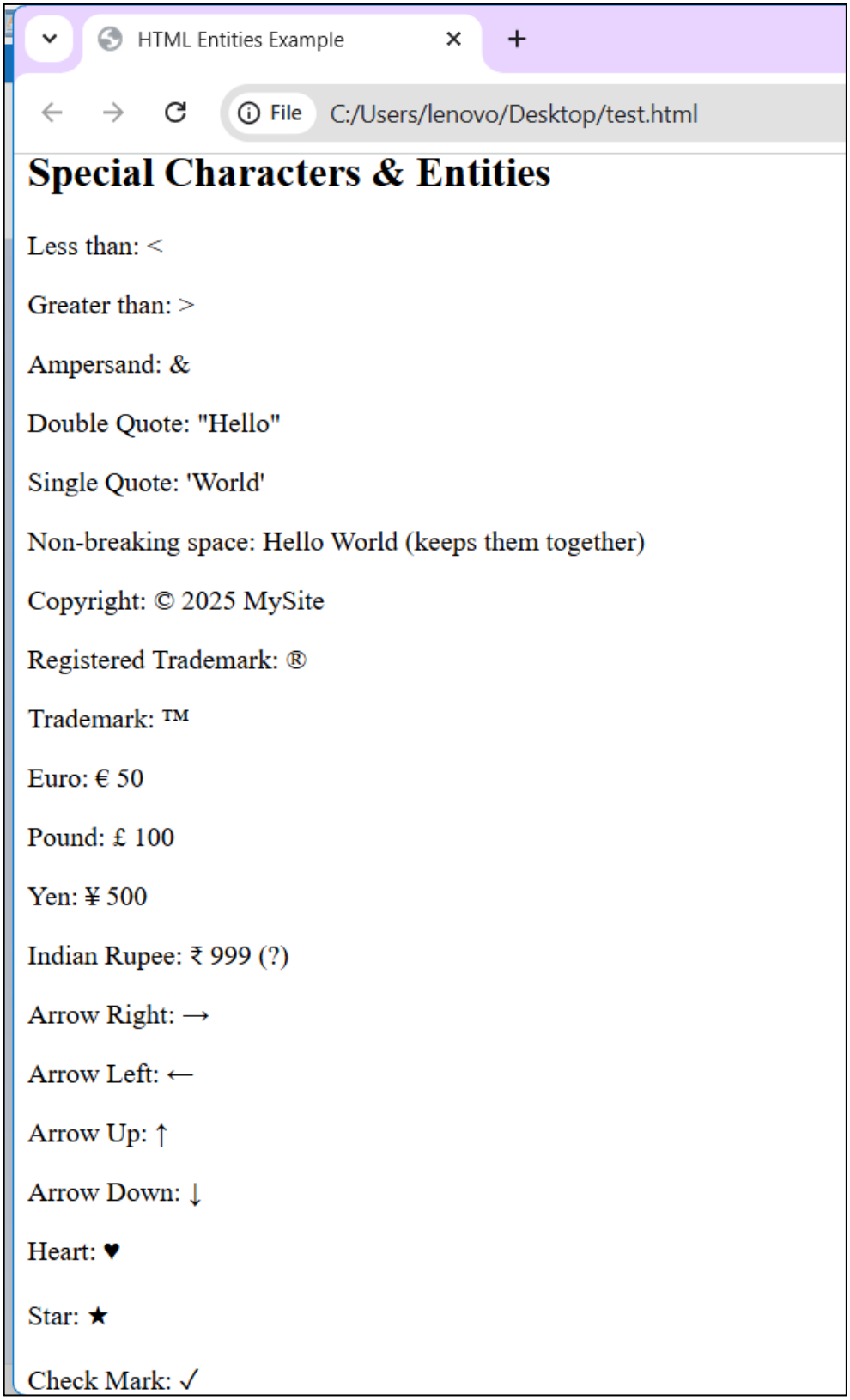
characters, such as:

- **ö** a lowercase o with an umlaut: ö
- **ñ** a lowercase n with a tilde: ñ
- **È** an uppercase E with a grave accent: È

NOTE: Unlike the rest of HTML, the escape sequences are case sensitive. You cannot, for instance, use < instead of <.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>HTML Entities Example</title>
  </head>
  <body>
    <h2>Special Characters & Entities</h2>
    <p>Less than: &lt;</p>
    <p>Greater than: &gt;</p>
    <p>Ampersand: &amp;</p>
    <p>Double Quote: &quot;Hello&quot;</p>
    <p>Single Quote: &apos;World&apos;</p>
    <p>Non-breaking space: Hello World (keeps them together)</p>
    <p>Copyright: &copy; 2025 MySite</p>
    <p>Registered Trademark: &reg;</p>
    <p>Trademark: &trade;</p>
    <p>Euro: &euro; 50</p>
    <p>Pound: &pound; 100</p>
    <p>Yen: &yen; 500</p>
    <p>Indian Rupee: &#8377; 999 (₹)</p>
    <p>Arrow Right: &rarr;</p>
```

Output:


HTML Entities Example
File
C:/Users/lenovo/Desktop/test.html
Special Characters & Entities
Less than: <
Greater than: >
Ampersand: &
Double Quote: "Hello"
Single Quote: 'World'
Non-breaking space: Hello World (keeps them together)
Copyright: © 2025 MySite
Registered Trademark: ®
Trademark: ™
Euro: € 50
Pound: £ 100
Yen: ¥ 500
Indian Rupee: ₹ 999 (?)
Arrow Right: →
Arrow Left: ←
Arrow Up: ↑
Arrow Down: ↓
Heart: ♥
Star: ★
Check Mark: ✓

1.14 Text related Elements(Quotation and Citation Elements)

HTML provides specific elements for semantically marking up quoted text and citations, enhancing accessibility and structure.

- **<abbr>** Defines an abbreviation or acronym**<bdo>** Defines the text direction
- **<blockquote>** Defines a section that is quoted from another source
- **<cite>** Defines the title of a work
- **<q>** Defines a short inline quotation

1.15 Using Multimedia (Image/video/Audio/youtube) in HTML:

A variety of tags such as the <img> tag, <video> tag, and <audio> tag are available in HTML to include media on your web page. Multimedia combines different media, such as images, audio, and videos. Users will have a better experience when multimedia is embedded into HTML.

Media Tag	Description
<audio>	An inline element is used to embed sound files into a web page.
<video>	Used to embed video files into a web page.
<source>	Used to attach multimedia files like audio, video, and pictures.
<embed>	Used for embedding external applications, generally multimedia content like audio or video, into an HTML document.<
<track>	Specifies text tracks for media components, specifically for audio and video elements.

```
<!DOCTYPE html>
<html lang="en">
 <head>
  <meta charset="UTF-8" />
  <title>Media Tags Example</title>
 </head>
 <body>
  <h1>Media Tags in HTML</h1>
  <h2>Image</h2>  <!-- Image Example -->
  <img src=https://via.placeholder.com/300x200 alt="Sample Image" width="300" height="200" />
  <p>This is an image using the <code>&lt;img&gt;</code> tag.</p> <!-- Audio Example -->

  <h2>Audio</h2>
  <audio controls>
   <source src="sample-audio.mp3" type="audio/mpeg" />
  <source src="sample-audio.ogg" type="audio/ogg" />
  Your browser does not support the audio element.
  </audio>

  <p>This is an audio player using the <code>&lt;audio&gt;</code>
  tag.</p> <h2>Video</h2> <!-- Video Example --> <video width="400"
  controls>

   <source src="sample-video.mp4" type="video/mp4" />
   <source src="sample-video.ogg" type="video/ogg" />
   Your browser does not support the video tag. </video>

  <p>This is a video player using the <code>&lt;video&gt;</code> tag.</p>
  <!-- Iframe Example -->
  <h2>Embedded Video (Iframe)</h2>
  <iframe width="400" height="250" src="https://www.youtube.com/embed/dQw4w9WgXcQ"
   title="YouTube Video" frameborder="0" allowfullscreen ></iframe>

  <p>This uses <code>&lt;iframe&gt;</code> to embed a YouTube video.</p>
</body> </html>
```

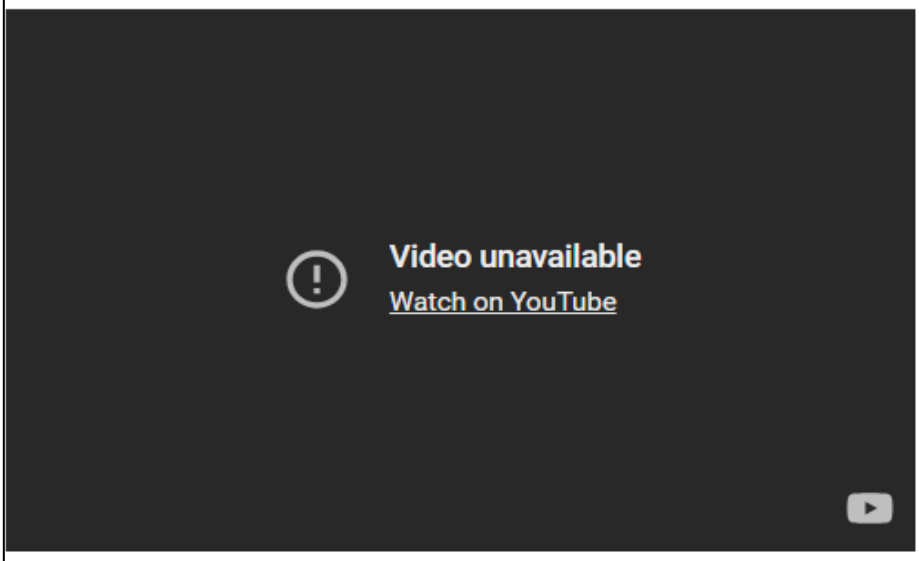
Output:


Media Tags Example
File
C:/Users/lenovo/Desktop/test.html
Media Tags in HTML
Image
This is an image using the <img> tag.
Audio
0:02 / 0:23
This is an audio player using the <audio> tag.

Video

This is a video player using the `<video>` tag.

Embedded Video (Iframe)



This uses `<iframe>` to embed a YouTube video.

Advantage of Media tag:

- Plugins are no longer required
- fashion than imported third-party
- Native (built-in) controls are provided by the browser.
- Accessibilities (keyboard, mouse) are built-in automatically

1.16 Creating & Using Hyperlinks:

The <a> HTML element (or anchor element), with its href attribute, creates a hyperlink to web pages, files, email addresses, locations in the same page, or anything else a URL can address.

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <title>hyperlink in HTML</title>
  </head>
  <body>
    <h1>Hyperlink in HTML</h1>
    <!-- Hyperlink Example -->
<a href="http://www.facebook.com" target= " _blank" > This is a link</a>
  </body>
</html>
```

Output:

Ms. Monika Singh – Notes + Practical (practicals are divided into sub-practicals (expandable format)

1.17 HTML Lists (Unordered, Ordered, Description, Nesting, Mixed)

1.18 Lists for Navigation Menus

1.19 Accessibility & Semantic Considerations

1.17 Introduction to HTML Lists

An HTML list helps present data on web pages in a structured manner, either in an ordered or unordered format, making the content easier to read and visually attractive. Lists play a key role in creating organized and accessible content in web development.

Types of HTML Lists

1. **Unordered Lists (<ul>):** These lists are used for items that do not need to be in any specific order. The list items are typically marked with bullets.

2. **Ordered Lists (<ol>):** These lists are used when the order of the items is important. Each item in an ordered list is typically marked with numbers or letters.

Attributes:

type → numbering style (1, a, A, i, I)
start → starting number/letter
reversed → reverses order

Example:

```
    <html>
</head>
<body>
Basic Example of HTML List
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport"
        content="width=device-width, initial-scale=1.0">
   <title>HTML</title>
</head>
<body>
   <h2>Welcome To ABES Engineering College </h2>
   <h5>List of available courses</h5>
   <ul>
        <li>Data Structures & Algorithm</li>
          <li>Web Technology</li>
          <li>Aptitude & Logical Reasoning</li>
         <li>Programming Languages</li>
 </ul>
 <h5>Data Structures topics</h5>
   <ol>
          <li>Array</li>
            <li>Linked List</li>
            <li>Stacks</li>
            <li>Queues</li>
            <li>Trees</li>
            <li>Graphs</li>
       </ol>
</body>
</html>
```

Output:

Basic Example of HTML List

Welcome To ABES Engineering College

List of available courses

- Data Structures & Algorithm
- Web Technology
- Aptitude & Logical Reasoning
- Programming Languages

Data Structures topics

1. Array
2. Linked List
3. Stacks
4. Queues
5. Trees
6. Graphs

3. Description Lists (<dl>): These lists are used to contain terms and their corresponding descriptions.

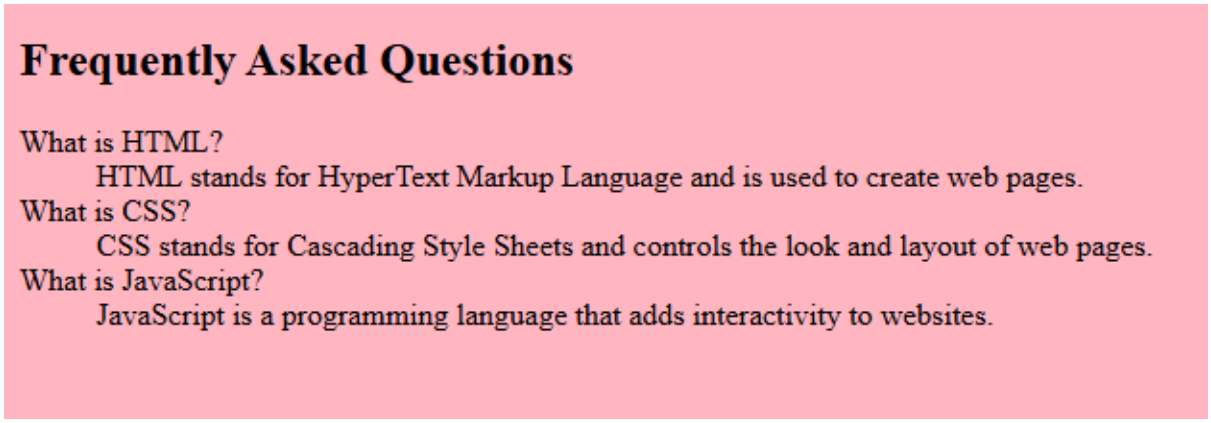
Contains:

1. <dt> → **Definition Term**
2. <dd> → **Definition Description**

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport"
    content="width=device-width, initial-scale=1.0">
    <title>HTML</title>
</head>
<body style="background-color: #FFB6C1";>
   <h2>Frequently Asked Questions</h2>
<dl>
  <dt>What is HTML?  </dt>
  <dd>HTML stands for HyperText Markup Language and is used to create web pages. </dd>
  <dt>What is CSS? </dt>
  <dd>CSS stands for Cascading Style Sheets and controls the look and layout of web pages.
</dd>
  <dt>What is JavaScript? </dt>
  <dd>JavaScript is a programming language that adds interactivity to websites. </dd>
</dl>
  </body>   </html>   </body>
</html>
```

Output:



4. Nesting lists- Nesting lists means placing one list **inside an item of another list**. The nested list is usually wrapped inside an <li> element of the parent list.

The purpose of Nesting List:

✓ To represent **hierarchical data** (e.g., categories and subcategories).
✓ To create **multi-level navigation menus**.
✓ To organize **outlines, directories, or structured content**.

Key Rules for Nesting Lists:

The nested list **must be inside an <li> element** of the parent list.

Nesting can be:

- **Unordered inside unordered** (<ul> in <ul>).
- **Ordered inside unordered** (<ol> in <ul>), and vice versa.

✓ Indentation (visually) helps users understand the hierarchy, but this is **done via CSS**, not by adding spaces in HTML.

Types of Nesting

1. **Unordered → Unordered** (Common in multi-level bullet points)
2. **Ordered → Ordered** (Common in outlines or steps with sub-steps)
3. **Mixed Nesting** (Combination of <ul> and <ol>)
4. **Nesting inside Description List** (less common, but possible inside <dd>)

Example of Nesting Lists:

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8">
   <meta name="viewport"
         content="width=device-width, initial-scale=1.0">
   <title>HTML</title>
</head>
<body style="background-color: #FFB6C1";>
<h3>Shopping List</h3>
<ul>
  <li>Fruits
    <ul>
      <li>Apple</li>
      <li>Mango</li>
      <li>Orange</li>
    </ul>
  </li>
  <li>Vegetables
    <ul>
      <li>Carrot</li>
      <li>Potato</li>
      <li>Tomato</li>
    </ul>
  </li>
</ul>
  </body>
</html>
```

Output:

5. **Mixed lists:** Mixed lists in HTML refer to the combination of **ordered lists (<ol>) and unordered lists (<ul>) within each other**. This means that an ordered list can contain an unordered list as a nested list or vice versa. The purpose of using mixed lists is to represent complex hierarchical information where some items need to be displayed in a specific order, while others do not.

   Mixed Lists uses:

   - **To organize information with different importance levels:** For example, main steps (ordered) and sub-options (unordered).
   - **To create structured outlines:** Where the top level is numbered, and the sub-points are bulleted.
   - **To maintain clarity in instructions, menus, and structured content.**

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>HTML</title>
</head>
<body style="background-color: #FFB6C1;">
    <h3>Web Development Topics</h3>
    <ol>
        <li>Frontend
            <ul>
                <li>HTML</li>
                <li>CSS</li>
                <li>JavaScript</li>
            </ul>
        </li>
        <li>Backend
            <ul>
                <li>Node.js</li>
                <li>PHP</li>
                <li>Python</li>
            </ul>
        </li>
        <li>Database
            <ul>
                <li>MySQL</li>
                <li>MongoDB</li>
            </ul></li>
    </ol>   </body>   </html>
```

Output:

1.18 Lists for Navigation Menus

Navigation menus are a crucial part of any website, allowing users to move between different pages or sections easily. In HTML, navigation menus are commonly created using unordered lists (<ul>) because the order of menu items is usually not important. These lists are typically placed inside a <nav> element, which provides semantic meaning, indicating that the section is for navigation purposes.

The best practice for creating navigation menus is to wrap each navigation link inside an <li> element, and then place all <li> elements within a <ul>. Each menu item is usually linked using the <a> (anchor) tag inside the list item. This structure ensures that the menu is both semantic and accessible, improving usability for screen readers and assistive technologies.

The uses of Navigation list:

Semantic Structure: Using <nav> with <ul> and <li> clearly defines the navigation section.

Accessibility: Screen readers can interpret lists properly, making navigation easier for visually impaired users.

Styling Flexibility: Lists provide an easy structure for applying CSS to create horizontal or vertical menus.

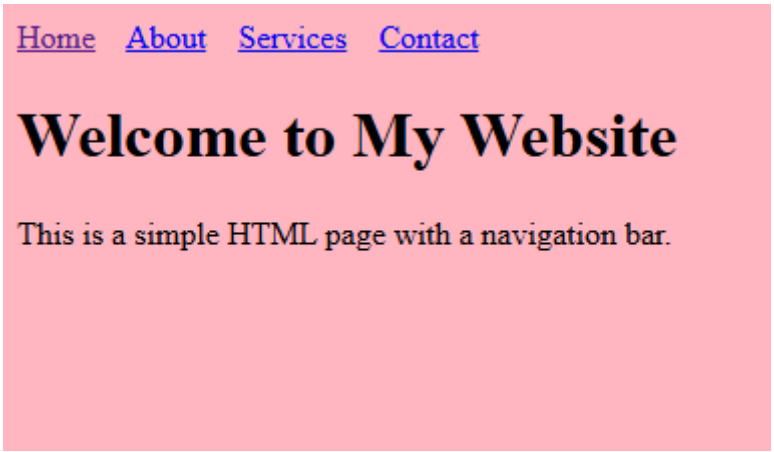
Consistency: Most browsers and frameworks support list-based menus, making it a standard practice.

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>HTML</title>
</head>
<body style="background-color: #FFB6C1;">
    <nav>
        <span><a href="index.html">Home</a></span>   
        <span><a href="about.html">About</a></span>   
        <span><a href="services.html">Services</a></span>   
        <span><a href="contact.html">Contact</a></span>
    </nav>

    <h1>Welcome to My Website</h1>
    <p>This is a simple HTML page with a navigation bar.</p>
</body>
</html>
```

Output:



1.19 Accessibility & Semantic Considerations

Semantic elements = elements with a meaning.

Accessibility considerations focus on designing websites for users of all abilities, while semantic HTML provides the structural meaning and context that assistive technologies like screen readers rely on to interpret content for visually impaired users and enable keyboard navigations.

When using lists in HTML, it is important to follow semantic and accessibility best practices to ensure that content is understandable and navigable for all users, including those using assistive technologies. **Semantic HTML means using the correct tags for their intended purpose:** for unordered lists, <ol> for ordered lists, <dl> for description lists, and <nav> for navigation menus. This not only improves readability for developers but also allows screen readers and other assistive tools to interpret the structure of the content accurately.

For navigation menus, wrapping menu items inside <li> elements within a <ul> and enclosing the whole menu in a <nav> element provides clear semantic meaning. Each link (<a> tag) should have descriptive text, avoiding generic phrases like “Click here,” so users understand the purpose of each link. Proper use of semantic tags helps screen readers announce the type and number of items in a list, enabling users to navigate efficiently.

Other accessibility considerations include maintaining sufficient color contrast, providing visible focus indicators for keyboard navigation, and avoiding excessive list nesting that may confuse users. Following these semantic and accessibility principles ensures that HTML lists are both meaningful and user-friendly across different devices and abilities.

Accessibility & Semantic Features in this Example: semantic glossary using a description list (<dl>). Each term (<dt>) like “HTML” or “CSS” is paired with its description (<dd>), making the relationship clear. Semantic tags help screen readers announce terms and definitions correctly, improving accessibility. Optional id attributes allow linking directly to specific terms. CSS can style the list for readability without affecting its structure, ensuring the glossary is both user-friendly and accessible.

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
</head>
<body style="background-color: #ffb6c1;">
    <h2>Web Development Glossary</h2>
    <dl>
        <dt id="html-term">HTML</dt>
        <dd>
            HyperText Markup Language, the standard language for creating web pages.
        </dd>
        <dt id="css-term">CSS</dt>
        <dd>
            Cascading Style Sheets, used for styling and layout of web pages.
        </dd>
        <dt id="js-term">JavaScript</dt>
        <dd>
            A programming language used to make web pages interactive.
        </dd>
        <dt id="api-term">API</dt>
        <dd>
            Application Programming Interface, a set of rules that allows software
            programs to communicate with each other.
        </dd>    </dl>
</body>   </html>
```

Output:

Web Development Glossary

HTML
: HyperText Markup Language, the standard language for creating web pages.

CSS
: Cascading Style Sheets, used for styling and layout of web pages.

JavaScript
: A programming language used to make web pages interactive.

API
: Application Programming Interface, a set of rules that allows software programs to communicate with each other.

Practical Use case:

1. **A company website lists its main navigation items like Home, About, Services, and Contact. Additionally, a product page lists key features using bullet points.**

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>HTML</title>
</head>
<body style="background-color: rgb(180, 233, 120);">
    <!-- Navigation Menu -->
    <h3>Company Navigation Menu</h3>
    <nav aria-label="Main Navigation">
        <ul>
            <li><a href="index.html">Home</a></li>
            <li><a href="about.html">About</a></li>
            <li><a href="services.html">Services</a></li>
            <li><a href="contact.html">Contact</a></li>
        </ul>   </nav>
    <!-- Product Features -->
    <h3>Product Key Features</h3>
    <div class="features">
        <ul>
            <li>High-speed performance</li>
            <li>Durable design</li>
            <li>Affordable price</li>
            <li>24/7 customer support</li>
        </ul>    </div>    </body>   </html>
```

Output:


Company Navigation Menu
Home About Services Contact
Product Key Features
High-speed performance
Durable design
Affordable price
24/7 customer support

2. **A cooking website lists a recipe where steps must be followed in order, such as mixing ingredients, baking, and serving.**

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
   <meta charset="UTF-8" />
   <meta name="viewport" content="width=device-width, initial-scale=1.0" />
   <title>HTML</title>
</head>
<body>
   <h1>Welcome to My Cooking Page</h1>
   <p>This page contains a simple recipe for baking a cake.</p>

   <h3>Recipe: How to Bake a Cake</h3>
   <ol>
      <li>Preheat the oven to 180°C (350°F).</li>
      <li>Mix flour, sugar, eggs, and butter in a bowl.</li>
      <li>Pour the batter into a greased baking tray.</li>
      <li>Bake for 25–30 minutes or until golden brown.</li>
      <li>Remove from oven and let it cool.</li>
      <li>Serve and enjoy!</li>
   </ol>
</body>
</html>
```

Output:


Welcome to My Cooking Page
This page contains a simple recipe for baking a cake.
Recipe: How to Bake a Cake
1. Preheat the oven to 180°C (350°F).
2. Mix flour, sugar, eggs, and butter in a bowl.
3. Pour the batter into a greased baking tray.
4. Bake for 25–30 minutes or until golden brown.
5. Remove from oven and let it cool.
6. Serve and enjoy!

3. **A web development tutorial defines terms like HTML, CSS, and JavaScript.**

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>HTML</title>
</head>
<body style="background-color: rgb(180, 233, 120);">
    <h3>Web Development Glossary</h3>
    <dl>
        <dt>HTML</dt>
        <dd>HyperText Markup Language, used to structure content on the web.</dd>
        <dt>CSS</dt>
        <dd>Cascading Style Sheets, used to style and layout web pages.</dd>
        <dt>JavaScript</dt>
        <dd>A programming language that makes web pages interactive.</dd>
        <dt>API</dt>
        <dd>Application Programming Interface, allows communication between
software programs.</dd>
    </dl>
</body>
</html>
```

Output:

Web Development Glossary

HTML
: HyperText Markup Language, used to structure content on the web.

CSS
: Cascading Style Sheets, used to style and layout web pages.

JavaScript
: A programming language that makes web pages interactive.

API
: Application Programming Interface, allows communication between software programs.

4. **An e-commerce website has a main category “Products” with subcategories “Electronics” and “Clothing.” Electronics further includes “Mobiles” and “Laptops.**

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>HTML</title>
</head>
<body style="background-color: rgb(180, 233, 120);">
    <h3>Product Categories</h3>
    <ul>
        <li>Products
            <ul>
                <li>Electronics
                    <ul>
                        <li>Mobiles</li>
                        <li>Laptops</li>
                    </ul>
                </li>
                <li>Clothing</li>
            </ul>
        </li>
        <li>About Us</li>
        <li>Contact</li>
    </ul>
</body>
</html>
```

Output:


Product Categories
Products
Electronics
Mobiles
Laptops
Clothing
About Us
Contact